

Governance Compliance und Security

Richtlinien, ISO-27001-Anwendbarkeit, Sicherheitsausnahmen, Qualitätsstandards

Der Regelrahmen der Plattform: Governance-Policies für Entwicklungskette, PR-Freigaben und Branch-Schutz, die ISO-27001 Statement of Applicability, akzeptierte Sicherheitsrisiken sowie Code-Quality- und QA-Standards. Hier steht, wer was freigeben darf und warum.

Repository: SvenKulesa/Finance

Stand: `claude/pdf-dokumente-notebook-llm-l6kcxq @ 4eadf9d`

Erzeugt am: 16.08.2026

Enthaltene Dokumente: 28

Umfang: ca. 17.683 Wörter

Dieses PDF ist eine Quelle für NotebookLM. Jedes eingebettete Dokument beginnt auf einer neuen Seite und nennt seinen Repository-Pfad, damit erzeugte Podcasts, Video-Skripte und Zusammenfassungen ihre Aussagen exakt auf die Originaldatei zurückführen können.

Inhalt dieses Quellpakets

1. Code Quality Standards & Architecture Directives	4
docs/code-quality/CODE_QUALITY_STANDARDS.md	
2. CAPITAL-AI ISO/IEC 27001:2022 Statement of Applicability	6
docs/compliance/ISO27001_STATEMENT_OF_APPLICABILITY.md	
3. DEVELOPMENT Chain Mutation Handoff Contract	18
docs/contracts/DEVELOPMENT_CHAIN_MUTATION_HANDOFF_CONTRACT.md	
4. Proposal: Disambiguate architecture `ADR-0014` phase documents	24
docs/governance/ADR_0014_ARCHITECTURE_PHASE_DISAMBIGUATION_PROPOSAL.md	
5. ADR Cross-Reference Inventory — Post Namespace Cleanup	26
docs/governance/ADR_CROSSREF_INVENTORY_2026-08-16.md	
6. ADR Unified Location Inventory — 2026-08-16	29
docs/governance/ADR_UNIFIED_LOCATION_INVENTORY_2026-08-16.md	
7. Autonomous Agent Roadmap Gate	30
docs/governance/AUTONOMOUS_AGENT_CONCEPT_GATE.md	
8. CAPITAL-AI Governance Library Report	36
docs/governance/CAPITAL_AI_GOVERNANCE_LIBRARY_REPORT_2026-08-15.md	
9. DEVELOPMENT Chain Branch Lifecycle Policy	38
docs/governance/DEVELOPMENT_CHAIN_BRANCH_LIFECYCLE_POLICY.md	
10. DevelopmentChain Documentation Freeze Policy	42
docs/governance/DEVELOPMENT_CHAIN_DOCUMENTATION_FREEZE_POLICY.md	
11. DEVELOPMENT Chain Execution Policy	44
docs/governance/DEVELOPMENT_CHAIN_EXECUTION_POLICY.md	
12. DEVELOPMENT Chain Responsibility Matrix	48
docs/governance/DEVELOPMENT_CHAIN_RESPONSIBILITY_MATRIX.md	
13. CAPITAL-AI Documentation Hygiene Policy	52
docs/governance/DOCUMENTATION_HYGIENE_POLICY.md	
14. GitHub-Actions-Budgetrichtlinie — 15 EUR/Monat	54
docs/governance/GITHUB_ACTIONS_BUDGET_POLICY.md	
15. CAPITAL-AI GitHub Main Protection Policy	57
docs/governance/GITHUB_MAIN_PROTECTION_POLICY.md	
16. GitHub Pro Enablement Decision — Documentation Refresh	60
docs/governance/GITHUB_PRO_ENABLEMENT_DECISION.md	
17. GROK Post-PR-Merge Continuation Policy	61
docs/governance/GROK_POST_PR_MERGE_CONTINUATION_POLICY.md	

18. GROK PR Project-Context Tagging Policy	64
docs/governance/GROK_PR_PROJECT_CONTEXT_TAGGING_POLICY.md	
19. HUMAN / OWNER Pull Request Approval Policy	68
docs/governance/HUMAN_OWNER_PR_APPROVAL_POLICY.md	
20. CAPITAL-AI Marketing Agent Roadmap Execution Policy	71
docs/governance/MARKETING_AGENT_ROADMAP_EXECUTION_POLICY.md	
21. Pull Request Check Classification	78
docs/governance/PR_CHECK_CLASSIFICATION.md	
22. Systemadmin Agent Roadmap Execution Policy	81
docs/governance/SYSTEMADMIN_AGENT_ROADMAP_EXECUTION_POLICY.md	
23. Work Claim — ADR-0014 Phase 3.4.3	88
docs/governance/work-claims/phase3-4-3-crypto-provider-chain.md	
24. README.phase3 4 3	89
docs/governance/work-claims/README.phase3-4-3.md	
25. . Test Plan & Quantitative Finance Engine (QA Perspective)	90
docs/qa/TEST_PLAN_AND_QA.md	
26. CAPITAL-AI — Supabase Free-Tier Leaked Password Protection Exception	93
docs/security/accepted-risk/SUPABASE_FREE_TIER_LEAKED_PASSWORD_PROTECTION_EXCEPTION_2026-08-02.md	
27. Developer Security Guidelines & Compliance Standards	96
docs/security/SECURITY_GUIDELINES.md	
28. CAPITAL-AI Security Remediation Baseline — 2026-08-12	98
docs/security/SECURITY_REMEDIATION_BASELINE_2026-08-12.md	

Code Quality Standards & Architecture Directives

Quelle: docs/code-quality/CODE_QUALITY_STANDARDS.md

Code Quality Standards & Architecture Directives

Project: CAPITAL-AI
Version: 0.6.0 (Beta-Phase)
Standard: TypeScript Strict Mode (`strict: true`)

General Code Integrity (Datenintegrität)

CAPITAL-AI maintains strict quality standards to ensure enterprise-level performance, clean refactors, and complete compliance with European regulations:

- Strict Type-Safety:** Avoid using `any`. Explicitly declare TypeScript interfaces and models for all parameters, components, and API responses.
- Defensive API Contracts:** All API responses must be validated upon receipt. Never assume any response is an array or object of correct shape without checking `Array.isArray()` or proper structural type guards. Handle exceptions gracefully without crashing components.
- No Legacy Versioning:** All references to deprecated version numbers are omitted. The entire platform is strictly pinned to **Version 0.6.0** (Beta-Phase) representing the current unified release.

Mandatory Coding Conventions

1. TypeScript Coding Rules

- Standard Enums:** Always declare enums using standard `enum` definitions rather than `const enum` to prevent compiler mapping mismatches across bundlers.
- Named Imports:** Always utilize named imports at the top-level of the file instead of full object destructuring on default imports.

```
// PREFERRED PATTERN
import { Sparkles, Shield, Award } from 'lucide-react';
import { SubscriptionTier } from '../types';
```

2. Modularity & Token Budget Constraints

To prevent generation cutoff and massive compile overhead: * **File Size Constraint:** Individual source files must never exceed **500 lines**. * **Extract Early:** Do not consolidate all logic into single files like `App.tsx` or `Screeners.tsx`. Move sub-components into `/src/components/`, common utility functions into `/src/utils/`, and static database definitions into

dedicated files. * **Shared Types:** Always export shared interfaces from `/src/types.ts` to prevent circular import trees.

3. Component Hook Safety (useEffect Guidelines)

- **Zero Body Updates:** Never write state mutators directly inside the component body.
- **Primitive Dependencies:** Dependency arrays in `useEffect`, `useMemo`, and `useCallback` must strictly use primitive values (strings, numbers, booleans) or heavily memoized objects.

```
// CORRECT PATTERN
useEffect(() => {
  let isCurrent = true;
  const loadContent = async () => {
    try {
      const response = await fetch(`/api/docs-file?path=${docPath}`);
      const data = await response.json();
      if (isCurrent) setContent(data.content);
    } catch (err) {
      console.error(err);
    }
  };
  loadContent();
  return () => {
    isCurrent = false; // Cleanup to prevent race conditions
  };
}, [docPath]); // docPath is a primitive string
```

Performance KPIs & Auditing

- **Render Latency:** Keep components light and eliminate redundant virtual DOM repaints.
- **Serverless Cold-Start Limit:** Maintain container coldstarts below **1.5 seconds** by using lazy-loaded imports for large client libraries like `Stripe` or `@supabase/supabase-js`.
- **Lighthouse Goals:** Target values for mobile and desktop are strictly **> 90 points** for performance, best practices, accessibility, and SEO.

CAPITAL-AI ISO/IEC 27001:2022 Statement of Applicability

Quelle: docs/compliance/ISO27001_STATEMENT_OF_APPLICABILITY.md

CAPITAL-AI ISO/IEC 27001:2022 Statement of Applicability

Document ID

COMP-SOA-0001

Bezug

ARCH-AUDIT-0002 (Enterprise FinTech Architecture Audit), Kapitel 4.6 (Compliance-Befund: „ISO 27001: 0 Treffer im Code, keine Control-Zuordnung“) und Kapitel 14.6 (12-Monats-Roadmap, J6 „ISO-27001-Zertifizierungsvorbereitung“).

Version

1.0.0

Status

Interne Vorbereitung — **kein Zertifizierungsnachweis**. Dieses Dokument ist die codebasis-verifizierte Grundlage für eine spätere externe Zertifizierung, ersetzt aber weder ein Independent Audit (siehe A.5.35 unten) noch die organisatorischen Entscheidungen, die eine tatsächliche Zertifizierung voraussetzt.

Prüfstichtag

2026-08-01

Methodik

Jede der 93 Controls aus ISO/IEC 27001:2022 Anhang A wird einzeln bewertet, mit einer von vier Kategorien:

Symbol	Bedeutung
Implementiert	Reale Code-/Konfigurations-Evidenz vorhanden, mit Fundstelle.
Teilweise	Ein Teilaspekt ist technisch abgedeckt, der Rest fehlt oder ist nicht verifizierbar.
Nicht implementiert	Kein Code-/Konfigurations-Nachweis, obwohl technisch anwendbar.
N/A / Unternehmensangabe	Control ist organisatorischer Natur (Personal, Verträge, physische Standorte, Managemententscheidungen) und aus dieser Codebasis grundsätzlich nicht ableitbar

Symbol	Bedeutung
	— keine Schätzung, keine Erfindung , konsistent mit der No-Demo-Data-Policy (docs/DATEN-SCHUTZ_PROTOKOLL.md). Wo die Cloud-Hosting-Architektur (Render, Supabase) ein physisches Control per Shared-Responsibility-Modell an den Anbieter delegiert, ist das vermerkt.

Wo die vorhandene Scanner-Zuordnung aus N6 (server/compliance/scanners.ts , Feld isoControls) einen technischen Nachweis liefert, wird die jeweilige Scanner-ID referenziert. Diese Auswertung ist eine **einmalige, manuelle Momentaufnahme** — sie aktualisiert sich nicht automatisch mit dem Code. Eine spätere Automatisierung (z. B. ein Coverage-Report direkt aus scanners.ts) ist in „Nicht Bestandteil dieses Dokuments" unten benannt.

Zusammenfassung

Kategorie	Anzahl	Anteil
Implementiert	21	23 %
Teilweise	28	30 %
Nicht implementiert	12	13 %
N/A / Unternehmensangabe	32	34 %
Gesamt	93	100 %

Von den **61 Controls, die aus dieser Codebasis grundsätzlich bewertbar sind** (93 abzüglich 32 rein organisatorischer/physischer bzw. anbieterverantworteter Controls), sind 21 vollständig und 28 teilweise technisch abgedeckt (80 % mit mindestens teilweiser Evidenz) — 12 haben keinen Nachweis. Diese Quote ist **kein Zertifizierungsstatus**: ISO 27001 zertifiziert das Informationssicherheits-Managementsystem (ISMS) einer Organisation, nicht eine Codebasis. Ein SoA-Dokument mit überwiegend technischer Evidenz ist eine notwendige, aber nicht hinreichende Voraussetzung.

A.5 Organisatorische Controls (37)

#	Control	Status	Evidenz / Begründung
A.5.1	Policies for information security		docs/DATEN-SCHUTZ_PROTOKOLL.md deckt Datenschutz ab; kein umfassendes ISMS-Policy-Set (Zugriffs-, Kryptografie-, Klassifizierungsrichtlinie als eigene Dokumente).
A.5.2	Information security roles and responsibilities		Wer intern welche Sicherheitsverantwortung trägt (CISO, Security Owner),

#	Control	Status	Evidenz / Begründung
			ist eine Organisationsentscheidung. Technische Rollen (<code>server/iam/types.ts</code> , <code>owner / admin / supervisor</code>) sind Zugriffsrollen, keine Sicherheitsverantwortlichkeiten.
A.5.3	Segregation of duties		IAM-Rollentrennung (<code>ADMIN_ZONE_ROLES</code> , <code>SUPERVISOR_ZONE_ROLES</code> , <code>OWNER_ONLY_ROLES</code> , <code>server/iam/types.ts</code>) trennt technische Zugriffssebenen; keine organisatorische Aufgabentrennung (z. B. Vier-Augen-Prinzip bei Deployments) dokumentiert.
A.5.4	Management responsibilities		Unternehmensangabe.
A.5.5	Contact with authorities		Unternehmensangabe.
A.5.6	Contact with special interest groups		Unternehmensangabe.
A.5.7	Threat intelligence		GitHub Dependabot Alerts sind aktiv (siehe A.5.21/A.8.8) — das ist automatisierte Schwachstellen-, keine Bedrohungsintelligenz im engeren Sinn. Kein dedizierter Threat-Intel-Prozess.
A.5.8	Information security in project management		Kein dokumentierter Sicherheits-Gate in der Projektplanung (ADRs dokumentieren Architektur-, nicht Sicherheitsentscheidungen als Prozess).
A.5.9	Inventory of information and other associated assets		<code>.ai/registry/ess-registry.json</code> + Komponenten-Manifeste (<code>src/platform/*/manifest.json</code>) katalogisieren Softwarekomponenten; kein Inventar der Datenbestände selbst (welche PII in welcher Tabelle).
A.5.10			Unternehmensangabe.

#	Control	Status	Evidenz / Begründung
	Acceptable use of information and other associated assets	✖	
A.5.11	Return of assets	✖	Unternehmensangabe (Offboarding-Prozess).
A.5.12	Classification of information	✖	Kein Klassifizierungsschema (öffentlich/intern/vertraulich) im Code oder in Docs.
A.5.13	Labelling of information		Keine Kennzeichnung, folgt aus A.5.12.
A.5.14	Information transfer	✖	TLS durch Render/Supabase (Plattform-Ebene); keine dokumentierte Data-Transfer-Vereinbarung (z. B. Auftragsverarbeitungsvertrag-Referenz).
A.5.15	Access control	✖	<code>server/iam/authMiddleware.ts</code> (<code>checkAdminAccess</code>), RLS-Policies in <code>supabase/migrations/</code> . Scanner: SEC-04, DAT-01.
A.5.16	Identity management		Supabase Auth + <code>profiles.iam_role</code> (<code>supabase/migrations/20260711000000_iam.sql</code>).
A.5.17	Authentication information	✖	TOTP-Secrets AES-256-GCM-verschlüsselt (<code>server/iam/secretCrypto.ts</code>), Passwort-Handling über Supabase Auth. Scanner: SEC-01.
A.5.18	Access rights	✖	Rollenbasierte Rechtevergabe (<code>server/iam/types.ts</code>), Step-Up-Erzwingung für Owner-Aktionen (D9). Scanner: SEC-05.
A.5.19	Information security in supplier relationships		Stripe-Webhook-Signaturprüfung (SEC-03) ist die einzige technisch verifizierte Lieferantenbeziehung; kein formales Supplier-Risk-Assessment für Supabase/Gemini/Alpha Vantage/NewsAPI.

#	Control	Status	Evidenz / Begründung
A.5.20	Addressing information security within supplier agreements		Vertragsinhalte sind Unternehmensangabe; technische Teilmenge siehe SEC-03.
A.5.21	Managing information security in the ICT supply chain	x	GitHub Dependabot Alerts aktiv (12 bekannte Schwachstellen zum Prüfstichtag laut GitHub-Meldung bei Pushes dieser Session) — Erkennung vorhanden, dokumentierter Behebungsprozess fehlt.
A.5.22	Monitoring, review and change management of supplier services	Δ	Kein dokumentierter Review-Zyklus für Drittanbieter-Änderungen (z. B. Supabase/Stripe API-Versionswechsel).
A.5.23	Information security for use of cloud services	Δ	RLS (Supabase), Docker-Deployment (Render) sind technische Bausteine; kein formaler Cloud-Security-Review-Prozess.
A.5.24	Information security incident management planning and preparation	x	docs/runbooks/DEPLOYMENT_ROLLBACK_UND_BACKUP.md (H7) deckt Deployment-/Datenvorfälle ab; kein breiterer Security-Incident-Response-Plan (z. B. Meldepflicht bei Datenpannen).
A.5.25	Assessment and decision on information security events	Δ	Kein formaler Triage-Prozess; security_events-Tabelle liefert Rohdaten, aber keinen dokumentierten Bewertungsworkflow.
A.5.26	Response to information security incidents	x Δ	Break-Glass-Mechanismus (break_glass_codes , supabase/migrations/20260731000400_security_events_st für Notfallzugriff vorhanden; kein allgemeiner IR-Playbook.
A.5.27	Learning from information security incidents		Kein dokumentierter Post-Incident-Review-Prozess.
A.5.28	Collection of evidence		iam_access_log , audit_logs_iam ,

#	Control	Status	Evidenz / Begründung
		 	uploads/sys-tem_events.json erfassen technische Nachweisspuren (D3); kein forensisches Verfahren für Beweissicherung.
A.5.29	Information security during disruption		H7-Runbook (Render-Rollback, Supabase-Backup-Verfahren).
A.5.30	ICT readiness for business continuity		H7-Runbook; keine dokumentierte/getestete DR-Übung.
A.5.31	Legal, statutory, regulatory and contractual requirements	 	docs/DATEN-SCHUTZ_PROTOKOLL.md (DSGVO-orientiert), No-Demo-Data-Policy. Finanzregulatorische Anforderungen (z. B. BaFin) sind NICHT durch echte Prüfung belegt — der vom Audit dokumentierte DEV-Fund (fabriziertes „BaFin“-Zertifikat, ARCH-AUDIT-0002 Kapitel 4.6) unterstreicht, dass hier keine ungeprüfte Behauptung stehen darf.
A.5.32	Intellectual property rights		Unternehmensangabe.
A.5.33	Protection of records		Stripe als Single Source of Truth für Billing-Records (BIL-02).
A.5.34	Privacy and protection of PII		PII-in-Logs-Scanner (DAT-04), docs/DATEN-SCHUTZ_PROTOKOLL.md. Kein vollständiges DPIA-Dokument.
A.5.35	Independent review of information security		ARCH-AUDIT-0002 selbst ist eine interne, KI-gestützte Prüfung — kein unabhängiges externes Audit im Sinne der Norm.
A.5.36	Compliance with policies, rules and standards for information security		21 automatisierte Scanner (server/compliance/scanners.ts) prüfen kontinuierlich technische Compliance-Aspekte; kein umfassendes Richtlinien-Compliance-Programm.

#	Control	Status	Evidenz / Begründung
A.5.37	Documented operating procedures		GOV-01/02/03-Scanner (ADR-/ESS-Registry-/Governance-Abdeckung), H7-Runbook, CI-Pipeline (D6).

A.6 Personal-Controls (8)

#	Control	Status	Evidenz / Begründung
A.6.1	Screening		Unternehmensangabe.
A.6.2	Terms and conditions of employment		Unternehmensangabe.
A.6.3	Information security awareness, education and training		Unternehmensangabe.
A.6.4	Disciplinary process		Unternehmensangabe.
A.6.5	Responsibilities after termination or change of employment		Unternehmensangabe.
A.6.6	Confidentiality or non-disclosure agreements		Unternehmensangabe.
A.6.7	Remote working		Step-Up-Authentifizierung + TOTP (D9) bilden eine technische Grundlage für sicheren Fernzugriff auf Admin-Zonen; keine dokumentierte Remote-Work-Richtlinie.
A.6.8	Information security event reporting		<code>security_events</code> -Tabelle + <code>system_events.json</code> bieten einen technischen Erfassungskanal; kein dokumentierter Melde prozess für Mitarbeitende.

A.7 Physische Controls (14)

Diese Anwendung läuft vollständig auf verwalteter Cloud-Infrastruktur (Render für Compute, Supabase für Datenbank). Physische Sicherheit der Rechenzentren liegt im Shared-Responsibility-Modell beim jeweiligen Anbieter — ein Nachweis erfordert deren eigene Zertifizierungen (z. B. Supabase/AWS SOC 2), nicht Code in diesem Repository.

#	Control	Status	Evidenz / Begründung
A.7.1	Physical security perimeters		

#	Control	Status	Evidenz / Begründung
			Anbieter-Verantwortung (Render/Supabase-Rechenzentren).
A.7.2	Physical entry		Anbieter-Verantwortung.
A.7.3	Securing offices, rooms and facilities		Anbieter-Verantwortung / Unternehmensangabe für eigene Büros.
A.7.4	Physical security monitoring		Anbieter-Verantwortung.
A.7.5	Protecting against physical and environmental threats		Anbieter-Verantwortung.
A.7.6	Working in secure areas		Anbieter-Verantwortung / Unternehmensangabe.
A.7.7	Clear desk and clear screen		Unternehmensangabe (Mitarbeiterverhalten).
A.7.8	Equipment siting and protection		Anbieter-Verantwortung.
A.7.9	Security of assets off-premises		Unternehmensangabe (z. B. Verschlüsselung von Mitarbeiter-Laptops).
A.7.10	Storage media		Anbieter-Verantwortung (Supabase verwaltet die physischen Speichermedien).
A.7.11	Supporting utilities		Anbieter-Verantwortung.
A.7.12	Cabling security		Anbieter-Verantwortung.
A.7.13	Equipment maintenance		Anbieter-Verantwortung.
A.7.14	Secure disposal or re-use of equipment		Anbieter-Verantwortung / Unternehmensangabe.

A.8 Technologische Controls (34)

#	Control	Status	Evidenz / Begründung
A.8.1	User endpoint devices		Unternehmensangabe (Endgeräte-/MDM-Richtlinie), keine App-Ebene.
A.8.2	Privileged access rights		server/iam/authMiddleware.ts, Step-Up-Erzwingung. Scanner: SEC-04, SEC-05.
A.8.3	Information access restriction		RLS (DAT-01), Quota-Durchsetzung (DAT-02),

#	Control	Status	Evidenz / Begründung
			verifizierte Identität in Billing (BIL-01).
A.8.4	Access to source code		GitHub-Repository-Berechtigungen liegen außerhalb dieser Codebasis (nicht durch Code verifizierbar); kein Nachweis über Branch-Protection-Konfiguration.
A.8.5	Secure authentication		TOTP (<code>server/iam/totp.ts</code>), Step-Up (D9), Supabase Auth.
A.8.6	Capacity management		Globales Rate-Limiting (S3, <code>server/iam/rate-limiter.ts</code>). Scanner: SEC-02.
A.8.7	Protection against malware		MIME-Type-Filter + 8 MB-Größenlimit für Uploads (D7, <code>server/ai.ts</code>); keine echte Signatur-/AV-Prüfung der Dateiinhalte.
A.8.8	Management of technical vulnerabilities		GitHub Dependabot Alerts aktiv (12 offene Meldungen zum Prüfstichtag); kein dokumentierter, terminierter Behebungsprozess.
A.8.9	Configuration management		Dockerfile (non-root USER , HEALTHCHECK , Q4), <code>render.yaml</code> , <code>npm run predeploy:check</code> (H7).
A.8.10	Information deletion		Keine dokumentierte Löschfrist/Retention-Policy für Nutzerdaten.
A.8.11	Data masking		Keine Maskierung sensibler Felder (z. B. in Logs oder Exporten) über die PII-Log-Prüfung (DAT-04) hinaus.
A.8.12	Data leakage prevention		PII-in-Logs (DAT-04), Secret-Hygiene (DAT-05).
A.8.13	Information backup		H7-Runbook dokumentiert das Supabase-Backup-Verfahren; die tatsächlich aktive Backup-Stufe ist laut Runbook selbst nicht programmatisch verifi-

#	Control	Status	Evidenz / Begründung
		■	zierbar (manuelle Betreiber-Prüfung nötig).
A.8.14	Redundancy of information processing facilities	■	Einzelinstanz-Deployment (<code>render.yaml</code> , ein <code>web</code> -Service), kein Multi-Region-/Failover-Setup.
A.8.15	Logging	▲	<code>server/logger.ts</code> (S4, Correlation-IDs), <code>iam_access_log</code> (D3).
A.8.16	Monitoring activities		<code>server/metrics.ts</code> (H6, Prometheus-Format), <code>GET /metrics</code> .
A.8.17	Clock synchronization	□	Serverzeit liegt bei Render/Supabase (NTP-Synchronisation Anbieter-Verantwortung); keine eigene Prüfung.
A.8.18	Use of privileged utility programs		Nicht anwendbar — keine systemnahen Utility-Programme in dieser Anwendungsarchitektur.
A.8.19	Installation of software on operational systems	▲	Docker-Image-basiertes, reproduzierbares Deployment statt manueller Software-Installation auf Produktivsystemen.
A.8.20	Networks security	▲	CORS-Konfiguration (SEC-06), TLS über Hosting-Plattform; keine eigene Netzwerksegmentierung (Single-Service-Architektur).
A.8.21	Security of network services	□	HTTPS über Render/Supabase; kein eigenes Service-Mesh oder mTLS zwischen Komponenten (nicht anwendbar bei Monolith).
A.8.22	Segregation of networks	■	Nicht anwendbar — Einzelservice-Architektur ohne interne Netzwerksegmente.
A.8.23	Web filtering		Nicht implementiert.
A.8.24	Use of cryptography		AES-256-GCM für TOTP-Secrets, SHA-256 für Token-Hashing (<code>server/iam/secretCrypto.ts</code>).

#	Control	Status	Evidenz / Begründung
A.8.25	Secure development life cycle		CI-Pipeline (D6), Testsuite (D5), <code>predeploy:check</code> (H7).
A.8.26	Application security requirements		Stripe-Webhook-Signatur (SEC-03), CORS (SEC-06), Security-Header (SEC-07, N7).
A.8.27	Secure system architecture and engineering principles		ADRs dokumentieren Architekturentscheidungen; kein formales Bedrohungsmodell (Threat Model) für die Anwendung als Ganzes.
A.8.28	Secure coding		TypeScript strict mode, SEC-01 (hartkodierte Secrets), QUA-01 (TODO/FIX-ME/HACK-Marker).
A.8.29	Security testing in development and acceptance		Vitest-Unit-Testsuite (128 Fälle zum Prüfstichtag); kein dediziertes SAST-/Penetrationstest-Tooling.
A.8.30	Outsourced development		Unternehmensangabe.
A.8.31	Separation of development, test and production environments		CI führt die Testsuite vor jedem Merge aus (Test-Gate); keine persistente Staging-Umgebung getrennt von Produktion (<code>render.yaml</code> definiert nur einen Service). Die im Audit dokumentierte Dev/Prod-Fork-Divergenz (ADR-0019) ist ausdrücklich KEINE Staging-Umgebung, sondern ein separater, unabhängiger Code-stand.
A.8.32	Change management		Git-PR-Workflow + CI-Pipeline (D6) als Änderungskontrolle vor jedem Merge nach <code>main</code> .
A.8.33	Test information		No-Demo-Data-Policy verbietet synthetische Daten im Produktionscode; Testdaten in <code>tests/unit/</code> sind synthetisch, aber eindeutig als Tests gekennzeichnet und isoliert.
A.8.34			Kein dokumentiertes Verfahren, das Produktivsys-

#	Control	Status	Evidenz / Begründung
	Protection of information systems during audit testing		teme während eines externen Audits/Pentests absichert.

Nicht Bestandteil dieses Dokuments

- **Kein Zertifizierungsantrag.** Eine tatsächliche ISO-27001-Zertifizierung erfordert einen akkreditierten externen Auditor (adressiert A.5.35 selbst), ein vollständiges ISMS mit Management-Commitment, sowie die Schließung der 22 „Unternehmensangabe“-Lücken oben — das liegt außerhalb dessen, was aus einer Codebasis ableitbar ist.
- **Keine automatisierte Coverage-Berechnung.** Diese Bewertung ist eine manuelle Momentaufnahme zum Prüfstichtag. Eine spätere Automatisierung (z. B. ein Report-Endpunkt, der die `isoControls`-Felder aus `server/compliance/scanners.ts` gegen diese Liste abgleicht und eine Coverage-Quote live berechnet) ist ein sinnvoller, aber separater Ausbauschritt.
- **Keine Behebung der 14 „Nicht implementiert“-Lücken.** Wie beim P0-Rechtsrisiko-Kapitel des Hauptaudits (ARCH-AUDIT-0002) ist die Ausweisung hier bewusst von der Umsetzung getrennt.

Verwandte Dokumente

- `docs/architecture/ENTERPRISE_FINTECH_ARCHITECTURE_AUDIT.md` (ARCH-AUDIT-0002), Kapitel 4.6, 14.6
- `server/compliance/scanners.ts` — technische Scanner mit `isoControls`-Zuordnung (N6)
- `docs/DATENSCHUTZ_PROTOKOLL.md` — No-Demo-Data-Policy, Grundlage für die Kennzeichnung statt Schätzung
- `docs/runbooks/DEPLOYMENT_ROLLBACK_UND_BACKUP.md` (H7) — Evidenz für A.5.24/29/30, A.8.13

DEVELOPMENT Chain Mutation Handoff Contract

Quelle: docs/contracts/DEVELOPMENT_CHAIN_MUTATION_HANDOFF_CONTRACT.md

DEVELOPMENT Chain Mutation Handoff Contract

Status: PROPOSED Version: 1.0.0 Date: 2026-08-12 Machine schema: .ai/contracts/development-chain-mutation-handoff.schema.json

Zweck

Der Mutation Handoff Contract übergibt einen bereits geplanten Roadmap-Mutationsschritt in einer begrenzten, maschinenlesbaren Form an einen autorisierten Execution Host oder Mutation Executor.

Der Contract ist ausdrücklich nicht autorisierend. Ein syntaktisch valider Handoff darf ohne separate Roadmap-/Owner-/REM-/IAM-/Audit-Autorität keinen Side Effect auslösen.

Trust Model

```
Roadmap / ESS / ADR / Runbook
  ↓
Human/Owner Approval Evidence
  ↓
REM / Agent IAM / Execution Host Policy
  ↓
M5 durable authorization evidence
  ↓
Mutation Handoff Contract
  ↓
exact side effect
  ↓
post-verification + outcome evidence
```

Der Handoff beschreibt den Auftrag; die Control Plane entscheidet, ob er ausführbar ist.

Pflichtfelder

Identität und Bindung

- `contractId`: eindeutige Work-Order-ID;
- `contractVersion`: Schema-/Contract-Version;
- `roadmapPhase`: z. B. M5A, M6, M7;
- `roadmapItem`: exakter Roadmap-Punkt;
- `repository`: immer der autorisierte Repository-Kontext;
- `baseBranch`;
- `baseSha`: exakter Baseline Commit;

- `owner` ;
- `executorAgentId` ;
- `authorityRefs` : ESS/ADR/Runbook/Roadmap;
- `approvalEvidenceRef` : separate Human Approval Evidence, sobald eine Mutation autorisiert wurde;
- die Approval Evidence bindet genau einen Administrator-Principal und einen benannten Roadmap-Abschnitt/Work Package.

Mutation

- `platform` : GITHUB , SUPABASE , STRIPE , RENDER , IONOS oder begründetes OTHER ;
- `mutationClass` ;
- `riskClass` ;
- `targetResource` : exakter Account/Project/Service/Environment/Resource;
- `allowedOperations` : enge Allowlist;
- `forbiddenOperations` : explizite Denylist;
- `expectedPostState` ;
- `idempotencyKey` ;
- `concurrencyKey` ;
- `expiresAt` .

Verification und Rollback

- `preMutationChecks` ;
- `postMutationChecks` ;
- `negativeChecks` ;
- `rollbackPlan` ;
- `evidencePath` ;
- `auditRequired` ;
- `dryRunRequired` .

Statusmodell

State	Bedeutung
DRAFT	Plan/Handoff wird erstellt; nicht ausführbar
ROADMAP_APPROVED	Architektur-/Roadmap-Paket akzeptiert; noch keine externe Mutation Approval
PRECHECK_PASS	Read-only Pre-Mutation Baseline passt; noch keine Side-Effect-Autorität
MUTATION_APPROVED	Separate Human/Owner Mutation Approval Evidence liegt vor
EXECUTING	

State	Bedeutung
	Execution Host hat Autorisierung/Audit gebunden und führt exakt den Auftrag aus
VERIFIED_PASS	Post-Verification und Evidence erfolgreich
FAILED_ROLLBACK_REQUIRED	Mutation oder Verifikation fehlgeschlagen; Rollback erforderlich
ROLLED_BACK	letzter verifizierter Zustand wurde wiederhergestellt und geprüft
CANCELLED	Auftrag wurde vor Ausführung beendet

Ein Statusfeld allein ist keine Autorisierung. Der Execution Host muss Approval-/REM-/Audit-Evidence separat verifizieren.

Capability-Regeln

Ein Handoff darf niemals folgende Capability erzeugen oder implizieren:

- MERGE ;
- Policy-/REM-Self-Expansion;
- Security-Control Disablement;
- Owner/Admin-Elevation;
- Secret Disclosure;
- beliebige/ungegrenzte Shell-Ausführung.

Reserved Human/Owner Mutation Classes bleiben Human-only, selbst wenn ein manipuliertes Dokument oder JSON sie als `allowedOperations` enthält.

Target Binding

`targetResource` muss so konkret sein, dass ein Executor den Zielzustand ohne freie Interpretation prüfen kann.

Beispiele:

- Render: exakter Service + Environment + erwarteter Commit/Deploy-Kontext;
- Supabase: exakter Project Ref + Auth/DB-Resource;
- Stripe: exakter Account/Mode + konkrete Webhook-/Config-Resource;
- GitHub: Repository + Base SHA + Branch/Ref/Workflow/Environment;
- IONOS: exakte Domain/Record-Ressource — bleibt grundsätzlich Human-reserviert.

Wildcards für produktionskritische Targets sind verboten.

Operation Binding

`allowedOperations` enthält semantische Operationen, keine ungefilterten Shell-Fragmente.

Beispiel:

```
[  
  "RENDER_UPDATE_ENV_KEY:FOO",  
  "RENDER_TRIGGER_DEPLOY:service-id"  
]
```

Ein Executor darf daraus keine zusätzlichen Operationen ableiten.

Secret Policy

Der Handoff darf enthalten:

- Secret-Namen/Identifiers;
- Credential Owner;
- Rotations-/Revocation-Referenzen.

Er darf nicht enthalten:

- Secret-Werte;
- API Keys;
- Passwörter;
- Tokens;
- TOTP Secrets/Codes;
- Passkey Private Keys;
- Recovery Codes.

Pre-Mutation Gate

Vor `MUTATION_APPROVED` sind mindestens zu prüfen:

1. aktuelles Target/Account/Environment;
2. Baseline/Head/Deployment Drift;
3. aktuelle Plattformgesundheit;
4. erforderliche Backups/Last-Known-Good;
5. Rollback-Ausführbarkeit;
6. kein konkurrierender Writer;
7. Approval-/Mandat-Gültigkeit;
8. erwartete Mutation bleibt innerhalb Risk/Scope.

Permit-before-side-effect

Wenn ein autonomer Executor eingesetzt wird:

1. Handoff parsen und Schema validieren;
2. REM/IAM/Reserved-Action-Policy prüfen;
3. exakte Human Approval Evidence prüfen;
4. durable Authorization Audit schreiben;

5. erst nach gültigem Audit Permit den Side Effect ausführen;
6. terminales `SUCCESS` oder `ERROR` append-only schreiben;
7. Post-Mutation Checks ausführen;
8. bei Fehler nach Runbook rollbacken.

Direkte Tool-/Connector-Mutation ohne diesen enforcebaren Pfad zählt nicht als autonom verifizierte DevelopmentChain-Ausführung.

Idempotenz und Replay

- jede Mutation erhält `idempotencyKey` ;
- ein bereits erfolgreich konsumierter Auftrag darf nicht erneut ausgeführt werden;
- `expiresAt` wird fail-closed geprüft;
- Base-/Target-/Approval-Drift invalidiert den Auftrag;
- Outcome-Evidence referenziert die Authorization Evidence;
- der Execution Host claimt `approvalEvidenceRef + idempotencyKey` atomar vor dem ersten Side Effect;
- derselbe Abschnitts-Grant ist nach Erfolg, Fehler, Abbruch oder Rollback verbraucht;
- Wiederholung, Parallelverbrauch oder Übertragung auf einen anderen Administrator-Principal ergibt `DENY_APPROVAL_REPLAY` ;
- ein erneuter Versuch benötigt eine neue Owner-Freigabe mit neuer Approval-Referenz und neuem Idempotency Key.

Evidence Output

Nach Abschluss müssen mindestens referenzierbar sein:

- `contractId` ;
- Roadmap Item;
- Authority/Approval References;
- Executor/Host/Actor;
- Target;
- Mutation Class/Risk;
- Precheck Result;
- Authorization Audit Reference;
- Outcome Audit Reference;
- Postcheck Result;
- Rollback State;
- Redacted technische Evidence;
- finaler DevelopmentChain State.

Relationship to REM

Ein Roadmap Execution Mandate (REM) ist die autorisierende Scope-Grenze des Systemadmin Executors. Dieser Handoff Contract ersetzt REM nicht und erweitert es nicht.

Bei Konflikt gilt die restriktivere Regel. Jede Abweichung führt zu **DENY/STOP**.

Proposal: Disambiguate architecture `ADR-0014` phase documents

Quelle: docs/governance/ADR_0014_ARCHITECTURE_PHASE_DISAMBIGUATION_PROPOSAL.md

Proposal: Disambiguate architecture `ADR-0014` phase documents

Document ID: GOV-PROP-ADR0014-ARCH-2026-08-16
Status: Proposal — requires Human/Owner decision
Related: ADR-0081, GOV-XREF-2026-08-16, ADR-0075

Problem

Two distinct concepts share the label **ADR-0014**:

Location	Meaning
<code>docs/adr/ADR-0014-documentation-governance-validator.md</code>	Canonical Documentation Governance Validator (ESS-0012)
<code>docs/architecture/**</code> files titled <code>ADR-0014 Phase 3.x...</code>	Server runtime / market-data decomposition work-stream

After PR #345, `docs/adr/` no longer has duplicate files for 0014, but the **architecture tree still claims the ADR-0014 identity** for a different decision family. Agents and humans can still resolve the wrong parent.

Options

Option A — Phase documents only (recommended default)

- Rename architecture titles from `ADR-0014 Phase ...` to `Phase 3.4.x – ...`
- Keep a single parent pointer, e.g. `Parent: docs/architecture/adr/ADR-0014-server-runtime-architecture-consolidation.md` **or** retitle that parent to drop the ADR number and call it `SERVER_RUNTIME_ARCHITECTURE_CONSOLIDATION.md`
- No new ADR numbers
- Historical work-claims keep path history; add a one-line “superseded title” note where needed

Pros: Minimal surface, no further ADR-number consumption.
Cons: Parent decision is not in the canonical `docs/adr/` registry.

Option B — Promote server-runtime parent into `docs/adr/`

- Assign **ADR-0082** (or next free) to Server Runtime Architecture Consolidation
- Point all Phase 3.4.x notes at ADR-0082

- Leave Documentation Governance as ADR-0014
- Optionally move or symlink the architecture parent file under `docs/adr/`

Pros: Single number space for real decisions.

Cons: More edits; work-claim and evidence path updates need Owner scope approval.

Option C — Status quo + warnings only

- Keep architecture filenames/titles
- Rely on ADR-0075 numbering notes and this inventory

Pros: Zero churn.

Cons: Ambiguity remains for every future agent pass.

Recommendation

Option A for phase notes immediately; consider **Option B** only if the server-runtime consolidation is still an active decision that should be re-audited as a first-class ADR.

Out of scope until Owner decides

- Mass rename of work-claim files under `docs/architecture/workclaims/`
- Changes to runtime code or CI
- Activation of ESS-0012 score thresholds

Decision request

Owner selects **A / B / C** (or hybrid). After selection, Grok opens a Class D PR implementing only the chosen option.

ADR Cross-Reference Inventory — Post Namespace Cleanup

Quelle: docs/governance/ADR_CROSSREF_INVENTORY_2026-08-16.md

ADR Cross-Reference Inventory — Post Namespace Cleanup

Document ID: GOV-XREF-2026-08-16
Basis: main after PR #345 / #348
Shadow-Score: 90/100 (advisory subset; 0 duplicate active numbers in docs/adr/)

1. Renumber map (PR #345)

Former active path under docs/adr/	New number
ADR-0014-phase-3-4-7-runtime-facade.md	ADR-0075
ADR-0020-version-manager-repository-convention-validator.md	ADR-0076
ADR-0044-canonical-naming-bilingual-documentation-vocabulary-governance.md	ADR-0077
ADR-0046-vocabulary-governance-authority-and-namespace.md	ADR-0078
ADR-0067-systemadmin-github-actions-execution-host.md	ADR-0079
ADR-0068-marketing-roadmap-executor-and-content-automation-boundary.md	ADR-0080

Canonical keepers remain: 0014 documentation-governance, 0020 multi-provider, 0044 production-runtime-immutability, 0046 modern-supabase, 0067 s1-security, 0068 first-bounded-autonomous. Decision record: **ADR-0081**.

2. Shadow-Validator result (post-merge)

Metric	Value
Active ADRs (docs/adr/)	72
Resolved	7
Drafts	2
Unique active numbers	72
Duplicate groups	0
Errors	0
Warnings	0
Info (missing Implementation-Status)	high density (expected; pre-existing)

3. Cross-reference debt

3.1 Parallel `ADR-0014` family under `docs/architecture/` — OPEN (Owner decision)

These files use the **ADR-0014** label for **server/market-data phase work**, while the canonical active ADR under `docs/adr/` for 0014 is **Documentation Governance Validator**:

Path	Topic
<code>docs/architecture/adr/ADR-0014-server-runtime-architecture-consolidation.md</code>	Server runtime consolidation
<code>docs/architecture/adr/ADR-0014-phase3-3-ai-analysis-extraction-note.md</code>	AI analysis extraction
<code>docs/architecture/adr/ADR-0014-phase3-4-market-data-adapters.md</code>	Market-data adapters
<code>docs/architecture/adr/ADR-0014-phase3-4-market-data-adapters-v2.md</code>	Market-data adapters v2
<code>docs/architecture/ADR-0014-PHASE-3.4.4.md</code>	Crypto/Stooq stages
<code>docs/architecture/ADR-0014-PHASE-3.4.6-MARKET-DATA-COMPATIBILITY-FACADE.md</code>	Compatibility facade
<code>docs/architecture/phase3-4-3-crypto-provider-extraction.md</code>	Crypto provider chain
Work-claims / evidence under architecture referencing “ADR-0014 Phase 3.4.x”	Phase tracking

Proposal: `docs/governance/ADR_0014_ARCHITECTURE_PHASE_DISAMBIGUATION_PROPOSAL.md`

Do **not** silently rewrite historical work-claim evidence paths without Owner review.

3.2 Internal reference in ADR-0075 — FIXED

Reworded: parent is the market-data compatibility refresh boundary under `docs/architecture/ADR-0014-PHASE-3.4.6-MARKET-DATA-COMPATIBILITY-FACADE.md`, with explicit note that `docs/adr/ADR-0014` remains Documentation Governance (ADR-0081).

3.3 `adr_history.json` — OPEN

Still incomplete relative to the live ADR tree (historical lag; known from maturity report). Separate maintenance PR.

4. Suggested next PRs (ordered)

1. ~~Score + inventory~~ — done (#348)
2. ~~ADR-0075 wording~~ — this PR
3. **Owner decision:** architecture `ADR-0014-*` phase disambiguation (see proposal)
4. **D/C:** backfill `Implementation-Status` on high-traffic active ADRs
5. Continue ESS-0012 engine work (not shadow)

5. Invariants

- Human/Owner-Gate and no agent self-merge unchanged.
- Shadow tools remain advisory (`exit 0` default).
- Full ESS-0012 Chapter-3 gate still **not** activated.

ADR Unified Location Inventory — 2026-08-16

Quelle: docs/governance/ADR_UNIFIED_LOCATION_INVENTORY_2026-08-16.md

ADR Unified Location Inventory — 2026-08-16

Document ID: GOV-XREF-UNIFIED-LOC-2026-08-16

Owner decision: Unified architecture ADRs at fixed location docs/adr/ only (Option B / hybrid)

Invariant

- Formal ADRs live **only** under docs/adr/ .
- Files under docs/architecture/ must **not** use ADR-XXXX as primary identity in filename or title for new work.
- Phase / work-package notes use PHASE-... or descriptive names and point to a parent ADR in docs/adr/ .

This change set

Action	Detail
Promote	Server Runtime Architecture Consolidation → ADR-0083
Demote	architecture ADR-0014 Phase 3.x → PHASE-3.x-... notes
Redirect	Former architecture ADR-0014 paths → short SUPERSEDED stubs
Fix	ADR-0075 parent → ADR-0083 + PHASE-3.4.6 path
README	Single-location rule added

Remaining debt (not in this PR)

Item	Note
docs/architecture/adr/ADR-0013-server-composition-root-...	Parallel number with canonical ADR-0013 (ESS). Follow-up: promote to unique ADR or rename to phase/ architecture note.
Work-claim / evidence paths that still say “ADR-0014 Phase ...”	Historical; optional one-line superseded notes later; no silent mass rewrite.
adr_history.json lag	Separate maintenance PR.

Next free ADR number after this PR

ADR-0084 (0082 = SeoEngine, 0083 = Server Runtime).

Autonomous Agent Roadmap Gate

Quelle: docs/governance/AUTONOMOUS_AGENT_CONCEPT_GATE.md

Autonomous Agent Roadmap Gate

Status: REQUIRED Date: 2026-08-12 Authority: ESS-0019, ESS-0021, ADR-0057, ADR-0058, ADR-0065, HUMAN_OWNER_PR_APPROVAL_POLICY.md

Purpose

Prevent ad-hoc creation or enablement of privileged autonomous/semi-autonomous agents while allowing low-risk daily automation through strictly read-only agents and a separately governed Systemadmin Roadmap Executor profile.

What "concept" means

In this DevelopmentChain, the required concept is the **Roadmap architecture package**. It is not an ad-hoc chat description.

Before a privileged or mutating agent is implemented, the roadmap package **MUST** be created from:

1. Deep Research / current best-practice comparison where relevant;
2. read-only inspection of the current Finance repository and existing production evidence;
3. gap analysis against the current DevelopmentChain state;
4. creation or update of the required ESS specification(s);
5. creation or update of the required ADR decision(s);
6. traceability and mutation/test gates;
7. rollback and kill-switch definition;
8. explicit Human/Owner approval of the resulting roadmap state.

Only after that package exists may implementation of a privileged agent begin.

Privileged agent rule

No autonomous or semi-autonomous agent may receive any of the following capabilities unless the roadmap package above exists and is Human/Owner-approved:

- BRANCH
- COMMIT
- PR
- CI_REQUEST
- DEPLOY_REQUEST

- PRODUCTION_MUTATION
- write access to Stripe, Supabase, Render or another production-connected platform
- credential rotation, role mutation, deployment trigger or billing mutation

`MERGE` remains prohibited as an agent capability.

The approved roadmap does not itself authorize a generic production mutation. Every mutation must remain inside the exact authority model defined by the approved roadmap, ADRs and the active mandate/policy.

Systemadmin Roadmap Executor profile

The only privileged profile that may operate from a standing Roadmap authorization instead of repeated per-PR creation prompts is:

`capital-ai-systemadmin-roadmap-executor`

Authority: ESS-0021, ADR-0065 and `SYSTEMADMIN_AGENT_ROADMAP_EXECUTION_POLICY.md`.

The profile **MUST** operate under an active Human/Owner-approved **Roadmap Execution Mandate (REM)**. The REM is a bounded, expiring, attributable authorization tied to one Roadmap segment, one repository/base branch, explicit paths/targets, explicit capabilities, maximum risk, mutation classes, tests, rollback, audit and kill switch.

A REM does not grant generic administrator authority.

Standing repository authorization

Within a valid REM, the Systemadmin profile **MAY** autonomously use the explicitly granted repository capabilities:

- READ
- ANALYZE
- PLAN
- BRANCH
- COMMIT
- PR
- CI_REQUEST

The agent may therefore execute a covered Roadmap work package from read-only analysis through a review-ready Pull Request without requesting a new Human authorization for each branch, commit or PR.

This standing authority is valid only while every action remains inside the mandate. Expiry, revocation, scope drift, target mismatch, self-elevation attempt or a material Roadmap change invalidates it and causes STOP.

External platform mutation

External production mutation is not implied by repository standing authority.

`PRODUCTION_MUTATION` may be delegated only after the Control Plane can technically enforce REM identity, exact target, mutation class, expiry, risk, preconditions, rollback and post-condition evidence. Until that capability is VERIFIED PASS, the Systemadmin Agent is autonomous only for repository/branch/PR mutations and explicitly non-production environments.

Non-delegable Owner boundary

The following remain Human/Owner-controlled under this profile:

- PR merge;
- weakening repository protections/Human gates;
- Owner/admin IAM elevation;
- Owner MFA reset/removal and break-glass;
- secret disclosure or unrestricted credential rotation;
- destructive production data operations;
- live billing money/entitlement mutation;
- production resource deletion;
- DNS/TLS/domain ownership change;
- disabling security/audit/RLS/consent controls;
- expansion or extension of the subject agent's own REM.

Daily-task read-only agent exception

Recurring or daily tasks MAY create or instantiate agents without prior Owner approval only when **all** of the following are true:

- allowed capabilities are limited to `READ` and `ANALYZE` ;
- no `BRANCH` capability;
- no `COMMIT` capability;
- no PR creation or update capability;
- no `CI_REQUEST` capability;
- no `DEPLOY_REQUEST` capability;
- no `PRODUCTION_MUTATION` capability;
- no `MERGE` capability;
- no write access to GitHub repository contents, Stripe, Supabase, Render or production configuration;
- credentials, if any, are read-only and least-privilege;
- output is informational/evidence/reporting only;
- retrieved/tool content cannot elevate privileges;

- logging/telemetry is enabled for the read-only profile;
- any transition from read-only to a mutating capability requires the full roadmap package and Human/Owner approval before capability elevation.

Examples include daily briefings, repository read-only audits, dependency/status observation and evidence collection. Creating or updating a pull request is explicitly outside this exception.

Required roadmap content for privileged agents

The roadmap package must define at minimum:

1. agent name, purpose and accountable human owner;
2. provider/client profile and runtime location;
3. exact allowed and prohibited capabilities;
4. target repositories/platforms/resources;
5. environment boundaries;
6. risk classification per capability;
7. human approval points;
8. production mutation points;
9. pre-mutation tests;
10. post-mutation verification;
11. telemetry/audit fields and evidence retention;
12. credential-holder model;
13. rollback, revocation and kill-switch procedure;
14. ESS references;
15. ADR references;
16. success/exit criteria and next roadmap phase after PASS.

For a Systemadmin Roadmap Executor, the package additionally **MUST** include or reference a machine-readable REM conforming to `docs/governance/ROADMAP_EXECUTION_MANDATE.schema.json` before enablement.

Approval sequence for privileged agents

Default privileged-agent sequence:

DEEP RESEARCH + REPO READ → ESS/ADR → ROADMAP DRAFT → OWNER REVIEW → OWNER APPROVED → IMPLEMENTATION PR → TECHNICAL VALIDATION → OWNER PR REVIEW → ENABLEMENT TEST → EVIDENCE → ENABLED

Systemadmin sequence after ESS-0021/ADR-0065 enablement:

DEEP RESEARCH + REPO READ → ROADMAP PACKAGE → OWNER APPROVES REM → SYSTEMADMIN EXECUTES COVERED WORK PACKAGES → PR → OWNER REVIEW/CI → HUMAN MERGE

No step may bypass mandate validation, Owner final PR review or merge authority.

Mutation restriction

For normal privileged agents, every production mutation still follows:

PRE-MUTATION TEST → HUMAN APPROVAL → MUTATION → POST-MUTATION VERIFICATION → EVIDENCE → NEXT PHASE

For the Systemadmin profile, a future technically enforced REM MAY represent the Human approval for a specifically enumerated bounded/reversible production mutation class. Until that enforcement is VERIFIED PASS, production mutation continues to require a separate Human approval exactly as before.

A failed or inconclusive verification results in STOP and, where defined, rollback.

Human-only transitions

The following remain Human/Owner-controlled unless a future separately approved ADR explicitly changes the model:

- approval of a roadmap package / REM for a privileged Systemadmin agent;
- PR merge authorization;
- non-delegable Owner actions listed above;
- transition to the next DevelopmentChain phase after a failed/inconclusive mutation/test gate;
- expansion or renewal of the Systemadmin REM.

Owner authentication / passkey note

The target security model SHOULD use WebAuthn/passkey step-up for privileged roadmap/REM approvals and production mutations, bound to actor + action + target + request/PR head where applicable. GitHub Actions cannot currently prove which physical device or passkey was used for a GitHub review; therefore repository workflows MUST NOT claim that a GitHub review alone proves device/passkey authentication.

A device identifier by itself is not accepted as a strong authentication factor. Until a CAPITAL-AI WebAuthn approval service exists, GitHub account identity `SvenKulessa` plus the current Human/Owner PR gate remains the enforceable repository evidence, and delegated production mutation remains disabled unless a separately accepted interim ADR provides equivalent assurance.

Provider neutrality

ChatGPT, Claude, Gemini/Google AI Studio and future clients are execution/development clients only. Provider/model identity grants no authority. NotebookLM and daily research agents remain read-only unless a later roadmap package explicitly elevates capabilities under Human/Owner approval.

Evidence

Each privileged agent roadmap package must be linked from `docs/architecture/ROADMAP.md` and the M0-M10 traceability matrix before implementation. The final enablement evidence records the approved roadmap version, ESS/ADR references, REM identifier where applicable, PR/commit SHA, test results, mutation evidence if any and the Owner approval reference.

Branch lifecycle

Privileged mutating work uses a fresh branch from current `main`. After successful merge into the Finance repository, the branch **MUST** be deleted. Closed/superseded work branches are removed after necessary Evidence retention. Merged branches are never reused for a new Roadmap item.

CAPITAL-AI Governance Library Report

Quelle: docs/governance/CAPITAL_AI_GOVERNANCE_LIBRARY_REPORT_2026-08-15.md

CAPITAL-AI Governance Library Report

Document ID: GOV-LIB-2026-08-15

Repo: SvenKulessa/Finance

Zweck: Kompakte, dauerhafte Bibliothek für Grok-Projektgedächtnis und Agenten-Kontext

1. Executive Summary

CAPITAL-AI betreibt eine **stark spezifizierte, fail-closed Multi-Agent-Governance** für eine produktive FinTech-/Investment-Webanwendung (`capital-ai.online`).

- Kernprinzipien (unverhandelbar):**
- 1. Technische CI $\neq$ Autorisierung zu PR-Erstellung oder Merge
 - 2. Human/Owner sieht Diff + Viewed + Review (`/ okay`) **vor** teurem `build-and-test`
 - 3. Strengste PR-Klasse (D < C < R < M) gilt für den ganzen PR
 - 4. Externe Plattformmutation braucht eigene Approval- und Evidence-Kette
 - 5. CI-Trust-Root darf sich nicht selbst bootstrafen (ADR-0069)

2. Schlüssel-ADRs (Auszug)

ADR	Thema
0039	Human-authorized PR creation; advisory Work-Claims
0047	GitHub authoritative pre-merge CI gate
0069	Human/Owner gate; kein selbstreferenzieller CI-Boots-trap
0073	CI consolidation <code>build-and-test</code>
0081	ADR namespace cleanup, shadow validator, weekly score

3. Operative Enforcement

- `.github/pull_request_template.md` v1.3.5
- `scripts/pr/classifyPrScope.mjs` , `validatePrBody.mjs`
- Owner-Gate vor teurer CI
- Shadow: `npm run governance:shadow-adr` / `npm run governance:score` (advisory)

4. Hinweis

Vollständige Analyse siehe ursprüngliche Grok-Bibliotheksfassung und `docs/architecture/GOVERNANCE_MATURITY_REPORT.md` . Dieser Repo-Pfad ist die versionierte Ankerkopie für Agents und Audits.

End of GOV-LIB-2026-08-15

DEVELOPMENT Chain Branch Lifecycle Policy

Quelle: docs/governance/DEVELOPMENT_CHAIN_BRANCH_LIFECYCLE_POLICY.md

DEVELOPMENT Chain Branch Lifecycle Policy

Status: PROPOSED Date: 2026-08-12 Repository: SvenKulesa/Finance Authority: DevelopmentChain, ADR-0039, ESS-0021/ADR-0065

Ziel

Diese Policy verhindert langlebige Agent-Branches, versehentliche Wiederverwendung gemergter Arbeitsstände, Direktarbeit auf `main` und unklare Zuständigkeit zwischen parallelen AI-/Human-Workstreams.

Verbindlicher Lifecycle

Jedes eigenständige Arbeitspaket verwendet:

```
current main
→ new scoped branch
→ bounded changes
→ Pull Request
→ Human review / required CI
→ explicit Human merge
→ remote branch deletion
→ local ephemeral workspace cleanup
```

1. Baseline

Vor Branch-Erstellung:

1. aktuellen `main` SHA auflösen;
2. offene PRs und Changed-File-Overlap prüfen;
3. Roadmap-/ESS-/ADR-Gate prüfen;
4. Checkklasse bestimmen;
5. Scope und erwartete Zielpfade festlegen.

Ein Branch darf nicht bewusst von einem veralteten Baseline-SHA erzeugt werden, wenn aktuelles `main` ohne Konflikt verfügbar ist.

2. Neuer Branch pro Workitem

Für jedes neue Roadmap-/Fix-/Dokumentations-Workitem ist ein **neuer Branch** erforderlich.

Erlaubte Beispiele:

- `agent/<scope>`
- `docs/<scope>`

- `fix/<scope>`

Ein bereits gemergter, geschlossener oder supersedeter Branch darf nicht für ein neues Workitem wiederverwendet werden.

3. Geklonte Repositories / Worktrees

Ein lokaler Clone, Agent-Workspace oder Worktree ist nur eine Arbeitskopie und niemals die Autorität.

Verbindlich:

- keine Commits direkt auf lokalem `main`;
- nach Clone/Checkout für jedes Workitem einen neuen Finance-Branch erzeugen;
- Remote-Ziel muss das autorisierte Finance Repository sein;
- kein Wechsel auf einen alten gemergten Agent-Branch für neue Arbeit;
- lokale temporäre Branch-/Worktree-/Clone-Zustände dürfen keine nicht gepushten Evidence-/Rollback-Informationen enthalten, bevor sie entfernt werden.

4. Parallel Work Gate

Vor dem ersten mutierenden Commit sowie erneut vor PR-Erstellung:

- offene PRs abrufen;
- Changed Files vergleichen;
- identische oder semantisch gekoppelte Zielpfade erkennen.

Bei Konflikt:

- `RESCOPE` ,
- `SEQUENCE` ,
- älteren Branch/PR zuerst abschließen,
- oder Human/Owner entscheidet ausdrücklich über die Konfliktbehandlung.

Ein Agent darf einen Konflikt nicht durch Force-Push, ungeprüftes Rebase oder Überschreiben fremder Arbeit „lösen“.

5. PR-Grenze

Der Branch bleibt bis zum Merge der einzige Schreibkontext für das Workitem.

Neue Commits nach Human Current-Head Review invalidieren die Review-/Approval-Evidence für den alten Head nach der jeweils geltenden Owner-Gate-Policy.

`MERGE` bleibt Human/Owner-only.

6. Nach erfolgreichem Merge

Nach erfolgreichem Human-Merge in `Finance/main` ist verpflichtend:

1. Merge SHA und finalen PR-Status in Evidence erfassen;
2. sicherstellen, dass keine erforderliche Post-Merge-Verifikation mehr vom Branch abhängt;
3. **Remote-Arbeitsbranch im Finance Repository löschen;**
4. lokalen Branch entfernen, sofern er nicht für Evidence-Untersuchung temporär benötigt wird;
5. kurzlebige, nur für das Workitem erzeugte Clone-/Worktree-Kopie nach Sicherung der Evidence löschen;
6. für das nächste Arbeitspaket erneut von aktuellem `main` starten.

7. Closed / Superseded / Abandoned

Nicht gemergte, geschlossene oder supersedete Branches werden nach Sicherung notwendiger Evidence ebenfalls gelöscht. Ein supersedeter Branch darf nicht als versteckte Fortsetzung eines neuen Roadmap-Punkts dienen.

8. Rollback

Repository-Rollback erfolgt grundsätzlich über einen **neuen** scoped Rollback-/Revert-Branch aus aktuellem `main`, nicht durch Wiederbelebung des ursprünglichen gemergten Arbeitsbranches.

Bei externer Plattformmutation gelten zusätzlich die plattformspezifischen Rollback-Schritte der DevelopmentChain.

9. Agentenregel

Alle AI-/Agent-Clients, einschließlich ChatGPT, Claude, Google AI Studio und des Systemadmin Roadmap Executors, müssen diese Branch-Regel befolgen.

Eine Execution Host Capability `BRANCH` autorisiert nur die Erzeugung des für den gebundenen Auftrag vorgesehenen neuen Branchs. Sie autorisiert keine Wiederverwendung oder Erweiterung eines alten Branchs.

Exit Evidence

Ein Roadmap-Schritt mit Repository-Mutation ist erst vollständig geschlossen, wenn folgende Zustände dokumentiert sind:

- PR merged by Human;
- final merge SHA bekannt;
- Branch deleted;
- ggf. Post-Merge/Production Verification abgeschlossen;

- Roadmap/Traceability synchronisiert.

DevelopmentChain Documentation Freeze Policy

Quelle: docs/governance/DEVELOPMENT_CHAIN_DOCUMENTATION_FREEZE_POLICY.md

DevelopmentChain Documentation Freeze Policy

Effective: 2026-08-11 Updated: 2026-08-12

1. The AI-Agent transformation is documentation-first.
2. M3-M9 code/config/infrastructure changes were prohibited until M2G Documentation Freeze became COMPLETE on `main`; M2G is now complete.
3. After M2G, DevelopmentChain implementation proceeds **sequentially by phase**. Documentation for later phases may be prepared in advance, but documentation readiness never unblocks a phase whose predecessor gate is incomplete.
4. Every DevelopmentChain step must update `docs/architecture/ROADMAP.md`, the detailed implementation roadmap, traceability matrix and affected ADR/ESS status.
5. The canonical operational index is `docs/roadmaps/DEVELOPMENT_CHAIN_ROADMAP.md`; the legacy `AI_AGENT_M0_M9_IMPLEMENTATION_ROADMAP.md` remains a stable-reference detail document.
6. Every phase with implementation or mutation must have the required execution/run-book, test, rollback, evidence and traceability contract before execution.
7. A PR that mixes unresolved architecture documentation with production mutation must be split.
8. Documentation-only PRs may contain documentation, `.ai` governance/contract metadata and agent directives, but no runtime/workflow/infrastructure side effect. They use Check Class D unless a stricter class is introduced by their actual changed files.
9. External production mutation requires the DevelopmentChain mutation sequence: Roadmap/ADR/ESS → Human/Owner approval → pre-mutation verification → authorized execution → post-verification → evidence → roadmap sync.
10. A mutation handoff is non-authorizing. `docs/contracts/DEVELOPMENT_CHAIN_MUTATION_HANDOFF_CONTRACT.md` cannot replace Human approval, REM/IAM, execution-host policy or durable audit evidence.
11. Every repository work item uses a fresh branch from current `main`. After successful Human merge into Finance, the work branch is deleted and never reused. Cloned repositories/worktrees follow the same rule and temporary copies are cleaned up after required Evidence is secured.
12. Parallel agent work is allowed only with explicit changed-file overlap inspection; conflicting writers must be sequenced or rescaled/rescoped.
13. M6-M10 implementation remains blocked by the sequential gates recorded in the DevelopmentChain Roadmap, even when their planning documents already exist.

14. `MERGE` remains Human/Owner-only; green CI, an execution permit or a mutation Handoff is never merge authorization.

Normative execution references:

- `docs/governance/DEVELOPMENT_CHAIN_EXECUTION_POLICY.md`
- `docs/governance/DEVELOPMENT_CHAIN_BRANCH_LIFECYCLE_POLICY.md`
- `docs/governance/DEVELOPMENT_CHAIN_RESPONSIBILITY_MATRIX.md`
- `docs/runbooks/DEVELOPMENT_CHAIN_PHASE_EXECUTION.md`
- `docs/traceability/DEVELOPMENT_CHAIN_DOCUMENT_TRACEABILITY_MATRIX.md`

DEVELOPMENT Chain Execution Policy

Quelle: docs/governance/DEVELOPMENT_CHAIN_EXECUTION_POLICY.md

DEVELOPMENT Chain Execution Policy

Status: ACTIVE

Date: 2026-08-12

Updated: 2026-08-16

Scope: CAPITAL-AI SvenKulessa/Finance

Authority: docs/architecture/ROADMAP.md , docs/roadmaps/

AI_AGENT_M0_M9_IMPLEMENTATION_ROADMAP.md , ESS-0019, ESS-0021, ADR-0039, ADR-0057..0066, ADR-0069 Nachtrag 2026-08-16

Zweck

Diese Policy definiert die verbindliche Ausführungslogik der CAPITAL-AI DEVELOPMENT Chain. Sie trennt Architektur-/Dokumentationsautorität, Repository-Implementierung, externe Plattformmutation, Verifikation und Human/Owner-Autorität voneinander.

Sie erteilt selbst **keine** Mutationsberechtigung. Jede konkrete Mutation benötigt die für den Roadmap-Punkt geltende ADR/ESS/REM-/Approval-Kette.

Stand 2026-08-16: Pre-CI-Owner-Gate mit Checkboxes und Review ☐ / okay ist **retired**. Merge bleibt Human/Owner-only. Ab **M10** Passkey/WebAuthn für CI-Autorisierung.

Kanonische Kette

```
READ-ONLY BASELINE
→ GAP / ROADMAP PACKAGE
→ ESS / ADR / RUNBOOK / TRACEABILITY
→ FRESH SCOPED BRANCH FROM CURRENT MAIN
→ REPOSITORY IMPLEMENTATION
→ PR / GOVERNANCE CHECKS / CI (ohne Checkbox-/Emoji-Gate)
→ HUMAN MERGE
→ BRANCH DELETE
→ READ-ONLY PRE-MUTATION CHECK (wenn externe Mutation erforderlich)
→ EXPLICIT OWNER MUTATION APPROVAL
→ NON-AUTHORIZING MUTATION HANDOFF
→ AUTHORIZED EXECUTION HOST / MUTATION EXECUTOR
→ POST-MUTATION VERIFICATION
→ APPEND-ONLY EVIDENCE
→ ROADMAP / TRACEABILITY SYNC
→ NEXT PHASE
```

Ab **M10** (nach VERIFIED PASS Cutover): vor teurer CI zusätzlich Passkey- AUTHORIZE_PR_CI laut M10-Runbook.

Ein Schritt darf nicht übersprungen werden, wenn er für den konkreten Roadmap-Punkt als **REQUIRED** markiert ist.

Grundprinzipien

1. **Roadmap vor Mutation.** Keine externe Plattformmutation ohne vorherige Roadmap-/ADR-/Runbook-Klassifikation.
2. **Sequenzielle Phasen.** M6-M10 bleiben geblockt, bis der jeweilige Vorgänger `VERIFIED PASS` ist.
3. **Fail closed.** Fehlende, abgelaufene, widersprüchliche oder nicht persistierbare Autorisierung führt zu `DENY/STOP`.
4. **Human Merge.** `MERGE` bleibt Human/Owner-only und wird keinem Agenten als Capability übertragen. Checkbox-/Emoji-Zeremonien sind keine Merge-Voraussetzung mehr.
5. **Keine Self-Authority.** Ein Agent darf REM, Capability-Grenzen, Owner-Gates, Audit-Controls oder Kill-Switches nicht zu seinen Gunsten erweitern.
6. **Evidence vor Statusfortschritt.** Ein Roadmap-Punkt wird erst nach positiver/negativer Verifikation und belastbarer Evidence geschlossen.
7. **Keine Secrets in Evidence.** Reusable Credentials, TOTP-Codes/Secrets, Passkey Private Keys, Biometriedaten, rohe Tokens und vollständige sensible Requests/Responses dürfen nicht persistiert werden.
8. **Ein Work Item = ein Branch = ein PR.** Branches werden nicht für nachfolgende Roadmap-Punkte wiederverwendet.
9. **Parallelität nur ohne Schreibkonflikt.** Aktive PRs/Branches werden vor Schreibarbeit auf Changed-File-Overlap geprüft.
10. **Transport ist keine Autorität.** ChatGPT Connector, Claude Tooling, Google AI Studio, MCP, SDK, GitHub Actions oder andere Hosts erhalten Autorität ausschließlich aus der Control Plane.

Rollen und Ausführungsgrenzen

Human / Owner

Behält mindestens:

- Architektur-/Roadmap-Freigabe bei sicherheitsrelevanten Entscheidungen;
- finale Merge-Autorität (explizite Anweisung; kein Agent-Self-Merge);
- ab M10: Passkey-Autorisierung für `AUTHORIZE_PR_CI` und privilegierte Step-ups;
- explizite Freigabe externer Produktionsmutationen;
- Owner/Admin-IAM-Elevation, MFA/Break-Glass und Recovery;
- Secret Disclosure/Rotation mit erweitertem Scope;
- destruktive Produktionsdatenoperationen;
- Live Billing/Money/Entitlement;
- Produktionsressourcen-Löschung;
- DNS/TLS/Domain-Ownership;
- Security-Control-Abschwächung.

Roadmap / Architecture / Documentation Plane

Darf read-only analysieren, Gaps klassifizieren, Roadmap-/ESS-/ADR-/Runbook-/Traceability-/Evidence-Vorgaben erstellen und Mutation Work Orders vorbereiten.

Development Implementation Plane — Google AI Studio

Google AI Studio ist die Entwicklungsumgebung für Anwendungscode, Architektur und Frontend. Externe Plattformmutationen (Stripe, Supabase, Render) werden dort nicht direkt ausgeführt.

Production Integration Plane — Claude

Claude ist für Produktionsüberführung und Backend-Konfiguration vorgesehen; Mutationsautorität bleibt an Roadmap, Owner Approval, Handoff und Audit gebunden.

Systemadmin / Mutation Executor

Nur innerhalb eines gültigen, Human/Owner-approved Mandats. Executor darf `MERGE` nicht ausführen und keine Reserved Human/Owner Actions über Handoff delegieren.

Parallel Work / Concurrent Writer Gate

Vor jedem neuen Schreib-Workitem: aktuellen `main` SHA, offene PRs/Changed Files und Zielpfade prüfen; bei Overlap `STOP/RESCOPE/SEQUENCE`.

Branch- und Clone-Lifecycle

```
current main → fresh scoped branch → commits → PR → Human merge → branch delete
```

Details: `docs/governance/DEVELOPMENT_CHAIN_BRANCH_LIFECYCLE_POLICY.md`.

PR-/CI-Klassifikation

Es gilt `docs/governance/PR_CHECK_CLASSIFICATION.md` (D/C/R/M). Owner-Checkbox-/Emoji-Gate ist retired.

Mutation State Vocabulary

`NOT REQUIRED` | `PLANNED` | `HUMAN APPROVED` | `MUTATED` | `VERIFIED PASS` | `FAILED / ROLLED BACK`

Mutation Handoff

Nicht autorisierender Arbeitsauftrag. Authority: Roadmap/ADR/ESS + Human/Owner Approval + REM / Agent IAM + M5 Audit.

Evidence Minimum

Baseline SHA, Authority refs, Branch/PR/Head/Merge SHA, Checkklasse, Mutation Class, Pre/Post Verification, Approval Evidence (wenn erforderlich; ab M10 Passkey-Evidence), Audit refs, Rollback State, Next Gate.

Stop- und Rollback-Regeln

STOP bei unerwartetem Target, Baseline-Drift, fehlender Approval Evidence, Open-PR-Overlap, fehlender Audit-Persistenz, fehlgeschlagenem Pre-Check, unbekanntem Side Effect, Post-Mutation FAILED/INCONCLUSIVE.

Phase-spezifische Runbooks

- M5A: docs/runbooks/M5A_SUPABASE_TOTP_AAL2_HARDENING.md
- M6: docs/runbooks/M6_SUPPLY_CHAIN_PROVENANCE.md
- M7: docs/runbooks/M7_DEPLOYMENT_IDENTITY_MUTATION.md
- M8: docs/runbooks/M8_AGENT_CUTOVER.md
- M9: docs/runbooks/M9_ASSURANCE_INCIDENT_BREAK_GLASS.md
- M10: docs/runbooks/M10_PASSKEY_OWNER_PR_AUTHORIZATION.md

Closure Rule

Eine Phase wird nur geschlossen, wenn Roadmap, detaillierte Roadmap, Traceability, betroffene ESS/ADR, Evidence und Mutation State konsistent sind. Der Abschluss eines PRs allein ist kein DevelopmentChain Exit Gate.

DEVELOPMENT Chain Responsibility Matrix

Quelle: docs/governance/DEVELOPMENT_CHAIN_RESPONSIBILITY_MATRIX.md

DEVELOPMENT Chain Responsibility Matrix

Status: PROPOSED Date: 2026-08-12

Zweck

Diese Matrix trennt Entscheidung, Planung, Implementierung, Mutation, Verifikation und Merge-Autorität. Provider- oder Tool-Namen sind keine Sicherheitsautorität; sie beschreiben nur die vorgesehene operative Rolle.

Rollen

Rolle	Hauptaufgabe	Schreib-/Mutationsgrenze
Human / Owner <code>SvenKulessa</code>	Roadmap-/Security-Entscheid, File Review, externe Mutation Approval, Merge	finale Autorität für reservierte High-Impact-Aktionen
ChatGPT Roadmap / Architecture Plane	Deep/read-only Analyse, Gap, Roadmap, ESS/ADR, Contracts, Traceability, Evidence-Design	keine implizite externe Produktionsmutation
Google AI Studio Development Plane	Anwendungscode, Architektur, Frontend, Tests, production-ready Dev-Implementierung	Stripe/Supabase/Render nur als Code/Handoff, nicht als direkte Plattformmutation
Claude Production Integration Plane	Production Handoff, Staging/Integration, Backend-Konfiguration	Mutation nur mit demselben Owner-/Control-Plane-Gate
<code>capital-ai-systemadmin-roadmap-executor</code>	bounded Repository-/Mutation-Workitems unter gültigem REM/Execution Host	kein MERGE; keine Self-Authority; Reserved Actions bleiben Human-only
GitHub Actions	reproduzierbare CI, Security-/Build-/Evidence-Ausführung; ggf. gebundener Execution Host	nur explizite Workflow-Permissions; kein allgemeines Adminrecht
Supabase	DB/Auth/Persistence Platform	Mutation nur Roadmap/Approval/Handoff/Verification
Stripe	Billing/Webhooks/Payments	Live Money/Entitlement bleibt Human-reserviert; technische Mutation nur explizit freigeben
Render	Hosting/Deployment Runtime	Deployment-/Config-Mutation nur M7/approved runbook
IONOS / DNS	Domain/TLS Ownership	Human/Owner-only, sofern kein späterer ADR gleichwertige Assurance einführt

RACI-ähnliche Zuordnung

Legende:

- **A** = Accountable / finale Autorität
- **R** = Responsible / führt aus
- **C** = Consulted / liefert Analyse/Design
- **V** = Verifier / unabhängige oder technische Prüfung
- **—** = keine Rolle

Aktivität	Owner	ChatGPT Roadmap	Google AI Studio	Claude Prod	Mutation Executor	GitHub CI
Read-only Repo-/Produktionsbaseline	A	R	C	C	C	—
Gap-/Roadmap-Definition	A	R	C	C	C	—
ESS/ADR/Threat Model/Runbook	A	R	C	C	C	—
Development-Code	A	C	R	C	R unter REM	V
Frontend-Code	A	C	R	C	R unter REM	V
Production-Handoff vorbereiten	A	R	C	R	C	—
External Pre-Mutation Check	A	C	—	C	R/V abhängig vom Host	—
Externe Mutation freigeben	A	—	—	—	—	—
Externe Mutation ausführen	A	—	—	R falls autorisiert	R falls autorisiert	R nur als gebundener Host
Post-Mutation Verification	A	C	—	R/C	R/C	V soweit automatisierbar
Evidence/Traceability Sync	A	R	C	C	C	V
PR File Review / Viewed	A/R	—	—	—	—	—
	A	—	—	—	—	

Aktivität	Owner	ChatGPT Roadmap	Google AI Studio	Claude Prod	Mutation Executor	GitHub CI
Expensive CI Authorization						R nach gültigem Gate
PR Merge	A/R	—	—	—	PROHIBITED	PROHIBITED
Branch Cleanup nach Merge	A	R/C	R/C	R/C	R für eigenen Branch	—

Development-vs-Production Boundary

Google AI Studio

Darf in der Development Plane:

- Anwendungscode und Architektur implementieren;
- Frontend entwickeln;
- Tests und production-ready Konfiguration im Repository vorbereiten;
- Kommentare/Runbooks/Handoffs für spätere Plattformkonfiguration erzeugen.

Darf nicht als Development-Aktion direkt:

- Supabase Produktionszustand verändern;
- Stripe Live-Zustand verändern;
- Render Produktionskonfiguration verändern.

Claude / Production Integration

Darf Produktionsüberführung und Backend-Integration vorbereiten bzw. ausführen, sofern der konkrete Schritt durch die DevelopmentChain autorisiert ist. Claude selbst ersetzt weder Human/Owner Approval noch REM/IAM/Audit/Execution-Host-Enforcement.

Mutation Executor

Der autonome Executor ist der bevorzugte bounded Ausführungspfad für Roadmap-Mutationen, sobald sein eigener Systemadmin Exit Gate **VERIFIED PASS** erreicht. Bis dahin ist seine Existenz oder ein offener Implementierungs-PR keine Mutationsautorität.

Human-reservierte Aktionen

Mindestens folgende Aktionen werden nicht durch eine DevelopmentChain Work Order delegiert:

- **MERGE** ;
- Branch-/Repository-Protection abschwächen;
- Owner/Admin-Elevation;
- Owner MFA/Passkey/Break-Glass Recovery;

- Secret Disclosure;
- unbeschränkte Credential Rotation;
- destruktive Produktionsdatenoperationen;
- Live Billing/Money/Entitlement;
- Produktionsressourcen-Löschung;
- DNS/TLS/Domain Ownership;
- Security/Audit/RLS/Consent deaktivieren;
- REM-/Policy-/Capability-Self-Expansion.

Concurrent Agent Rule

Mehrere AI Apps/Agents dürfen parallel arbeiten, wenn:

1. jede Schreibaufgabe einen eigenen neuen Branch besitzt;
2. Changed-File-Scopes vor Beginn und vor PR-Erstellung verglichen werden;
3. keine nicht gemergte Datei eines anderen PRs als normative Voraussetzung angenommen wird;
4. bei Überschneidung sequenziert oder rescopet wird;
5. der Human/Owner die finale Merge-Reihenfolge bestimmt.

Evidence Responsibility

Der Ausführende erzeugt technische Evidence; der Roadmap/Documentation Plane synchronisiert die kanonischen Dokumente; der Owner akzeptiert keine Phase als abgeschlossen, bevor Mutation/Test/Evidence **VERIFIED PASS** und konsistent sind.

CAPITAL-AI Documentation Hygiene Policy

Quelle: docs/governance/DOCUMENTATION_HYGIENE_POLICY.md

CAPITAL-AI Documentation Hygiene Policy

Status: DRAFT IMPLEMENTATION Date: 2026-08-10 Scope: H0 + H4 + H5

Deutsch

Diese Policy definiert die kanonische Dokumentationshygiene fuer CAPITAL-AI.

Root Policy

Im Repository-Root sind nur explizit zugelassene Markdown-Dokumente erlaubt:

- README.md
- AGENTS.md
- CLAUDE.md

Neue fachliche, architektonische, operative, Compliance- oder Evidence-Dokumente muessen unter docs/ in einem fachlich passenden Unterordner abgelegt werden. Historische Evidence wird archiviert statt geloescht.

Canonical folders

- docs/architecture/ — Architektur, Roadmaps, technische Entscheidungen und Integrations-Evidence.
- docs/adr/ — Architecture Decision Records.
- docs/compliance/ — Compliance-, Datenschutz- und regulatorische Dokumentation.
- docs/governance/ — Governance Policies, Registry und Dokumentationsregeln.
- docs/traceability/ — Traceability-Reports und Coverage Evidence.
- docs/archive/ — historische oder superseded Evidence, sofern nicht bereits in einem fachlichen Archivpfad abgelegt.

Document identity

Kanonische Dokumente werden ueber eine stabile documentId identifiziert. Der Dateipfad ist ein veraenderbares Attribut und keine Identitaet. Registry-Eintraege enthalten mindestens Typ, Owner, Authority, Version, Sprache, Lifecycle-Status und Pfad.

CI hygiene gate

H5 wird innerhalb des bestehenden Unit-Test-Laufs ausgefuehrt. Es wird kein zusaetzlicher GitHub-Actions-Workflow eingefuehrt. Der Gate prueft mindestens:

1. keine nicht freigegebenen Root-Markdown-Dateien;
2. eindeutige Document IDs;

3. eindeutige Registry-Pfade;
4. vorhandene Registry-Zieldateien;
5. gueltige Lifecycle- und Language-Werte.

Damit bleibt die bestehende Actions-Budgetrichtlinie unveraendert.

English

This policy establishes the canonical documentation hygiene rules for CAPITAL-AI. Only `README.md`, `AGENTS.md` and `CLAUDE.md` are allowed as root-level Markdown documents. Domain documentation belongs under `docs/` in canonical folders. Canonical documents use stable document IDs; paths are mutable metadata rather than identity. The H5 hygiene gate runs inside the existing unit-test job and therefore does not add another GitHub Actions pipeline.

GitHub-Actions-Budgetrichtlinie — 15 EUR/Monat

Quelle: docs/governance/GITHUB_ACTIONS_BUDGET_POLICY.md

GitHub-Actions-Budgetrichtlinie — 15 EUR/Monat

Status: Verbindlich

Geltungsbereich: SvenKullessa/Finance, alle menschlichen und KI-basierten Contributors (einschließlich ChatGPT, Claude und weiterer Agenten)

Budget: maximal 15 EUR GitHub-Actions-Zusatzkosten pro Kalendermonat

1. Ziel und Grundsatz

GitHub Actions ist eine begrenzte CI-Ressource. Jeder Contributor MUSS Testläufe minimieren und vorhandene CI-Evidence wiederverwenden. Ein neuer Commit, Rebase, Branch-Sync oder manueller Re-Run darf nicht allein dazu dienen, einen bereits grünen Check erneut auszuführen.

Das GitHub-Budget von 15 EUR ist eine Obergrenze und kein Verbrauchsziel. Die Pipeline ist so auszulegen, dass sie primär innerhalb des im GitHub-Plan enthaltenen Freikontingents arbeitet.

2. Verbindliche Run-Limits

Pull Request

- Pro PR-Head-SHA ist genau **ein vollständiger technischer Linux-Testlauf** zulässig.
- Ein zusätzlicher Linux-Job ist nur für eine spezialisierte, nicht duplizierende Sicherheits-/Governance-Prüfung zulässig.
- Maximales reguläres Runner-Budget je PR-Head-SHA: **30 Linux-Runner-Minuten**.
- Nach einem fehlgeschlagenen Lauf ist genau **ein gezielter Wiederholungslauf nach einer tatsächlichen Korrektur** zulässig.
- `Re-run all jobs` ohne Code-/Konfigurationsänderung ist untersagt, außer bei dokumentierter GitHub-/Runner-Infrastrukturstörung.
- Mehrere kleine Fix-Commits sollen lokal oder durch vorhandene Evidence gesammelt validiert werden; nicht jeder Zwischenstand soll einen neuen Remote-CI-Lauf erzeugen.

main

- Pro Merge-SHA ist genau **ein vollständiger Produktions-CI-Lauf** zulässig.
- Der Render-Deploy darf nur aus diesem erfolgreichen `main`-Lauf für den exakt geprüften SHA ausgelöst werden.
- Kein zusätzlicher GitHub-Actions-Lauf darf nur zur Deployment-Verifikation gestartet werden.

Dokumentations-/Metadatenänderungen

- Änderungen ausschließlich an `docs/**`, Markdown-Dateien oder nicht ausführbaren Repository-Metadaten sollen keinen vollständigen Node-/Docker-Testlauf auslösen, sofern keine Workflow-, Security-, Runtime- oder Deployment-Invariante betroffen ist.

3. Verbotene Kostenmuster

Folgende Muster sind nicht zulässig:

1. identische `npm ci`, TypeScript-, Unit-Test- und Production-Build-Sequenzen in mehreren Workflows für denselben PR-SHA;
2. vollständiger Docker-Build in mehr als einem Workflow für denselben SHA;
3. manuelle Re-Runs zur Statuskosmetik;
4. Branch-Synchronisierung ohne tatsächliche `main`-Drift;
5. automatisches Polling durch Agenten, das neue Actions-Runs auslöst;
6. Matrix-Builds, macOS- oder Windows-Runner ohne explizite Architekturentscheidung;
7. größere GitHub-hosted Runner ohne explizite Freigabe;
8. CI-Commits, deren einziger Zweck die erneute Auslösung unveränderter Tests ist.

4. Docker-Regel

Der vollständige Docker-Image-Build ist teuer und wird deshalb selektiv ausgeführt:

- immer auf `main` vor dem Render-Deploy;
- auf Pull Requests nur, wenn Docker-/Runtime-/Dependency-/Deployment-relevante Dateien verändert wurden, insbesondere `Dockerfile`, `.dockerignore`, `package.json`, `package-lock.json`, `server/**`, Runtime-Guards oder Docker-Security-Skripte;
- nicht für reine UI-, Text- oder Dokumentationsänderungen, sofern keine Runtime-Invariante betroffen ist.

Die statische Docker-Hardening-Policy darf weiterhin als kostengünstiger Bestandteil des normalen technischen Gates laufen.

5. Budgetsteuerung

GitHub Billing MUSS mit einem monatlichen Actions-Budget von maximal **15 EUR** und aktivierter Stop-Regel am Budgetlimit betrieben werden. Empfohlene Warnschwellen:

- 50 %: Kostenprüfung;
- 75 %: keine optionalen Remote-Testläufe mehr;
- 90 %: nur merge-/produktionskritische CI;
- 100 %: Hard Stop; keine Budgeterhöhung ohne bewusste Owner-Entscheidung.

Ein KI-Agent darf das Budget, Spending Limit oder Billing-Konfigurationen niemals selbst erhöhen.

6. Agentenrichtlinie für ChatGPT, Claude und andere KI-Systeme

Vor jedem Commit oder PR-Update MUSS ein Agent:

1. prüfen, ob der aktuelle PR-Head bereits valide CI-Evidence besitzt;
2. Änderungen bündeln, statt mehrere CI-auslösende Mikro-Commits zu erzeugen;
3. keinen manuellen Re-Run auslösen, wenn kein technischer Grund besteht;
4. bei einem CI-Fehler zuerst Logs/Root Cause analysieren und erst danach einen korrigierenden Commit erzeugen;
5. vorhandene grüne Evidence nicht durch unnötiges Rebase/Synchronisieren invalidieren;
6. bei absehbarer Überschreitung der Limits den Owner informieren und keine zusätzliche kostenpflichtige CI auslösen.

7. Pipeline-Zielbild

```
Pull Request
-> 1x Build/Test-Gate
-> nur betroffene Spezial-Gates
-> kein Production Deploy

Merge nach main
-> 1x vollständiges Production-Gate
-> 1x Docker-Image-Build
-> Render Deploy Hook für exakt geprüften SHA
```

PR Technische Validierung darf künftig keine zweite vollständige Kopie des normalen CI-Gates darstellen. Doppelte Test-/Build-Arbeit ist zu entfernen oder in einen einzigen wiederverwendbaren Workflow zu konsolidieren.

8. Ausnahmeprozess

Eine Überschreitung der Run-Limits ist nur zulässig bei:

- GitHub-Infrastrukturfehlern;
- sicherheitskritischen Incident-Fixes;
- notwendiger reproduzierbarer Fehleranalyse, die lokal nicht möglich ist.

Die Ausnahme MUSS im PR mit Grund und zusätzlicher Run-Anzahl dokumentiert werden.

9. Review

Die Richtlinie wird überprüft, sobald einer der folgenden Fälle eintritt:

- GitHub-Plan oder Actions-Preise ändern sich;
- das monatliche Budget von 15 EUR wird zweimal innerhalb von drei Monaten zu mehr als 75 % ausgeschöpft;
- ein Self-hosted Runner eingeführt wird;
- die CI-Architektur wesentlich geändert wird.

CAPITAL-AI GitHub Main Protection Policy

Quelle: docs/governance/GITHUB_MAIN_PROTECTION_POLICY.md

CAPITAL-AI GitHub Main Protection Policy

Status: M1 Baseline Stand: 2026-08-10 Repository: `SvenKulesa/Finance`

Ziel

Diese Policy definiert den verbindlichen GitHub-Layer der providerunabhängigen AI-Agent-Delivery-Kette. Sie ergänzt CAPITAL-AI IAM, Capability Grants, Step-up, Approval und Audit Controls; sie ersetzt diese nicht.

Verifizierter Serverzustand

Das aktive Ruleset `main-production-protection` schützt `main` bereits durch:

- Pull-Request-Pflicht,
- Required Check `build-and-test`, strict,
- Löschschutz,
- Non-Fast-Forward-Schutz,
- keine Bypass Actors.

Single-Owner-Ausnahme

Zum M1-Zeitpunkt besitzt das private Repository genau einen Collaborator mit Admin-Rechten: `SvenKulesa`.

Deshalb bleibt `required_approving_review_count = 0`, solange keine zweite unabhängige vertrauenswürdige Reviewer-Identität existiert. Eine pauschale Pflicht zu einer GitHub-Approval würde den PR-Autor im aktuellen persönlichen Repository dauerhaft blockieren und damit kein wirksames Vier-Augen-Prinzip erzeugen.

Bis zur Einführung einer unabhängigen Reviewer-Identität gelten als kompensierende Kontrollen:

1. kein Agent darf direkt nach `main` schreiben;
2. Agentenänderungen müssen über isolierte Branches und Pull Requests laufen;
3. `build-and-test` bleibt fail-closed;
4. HIGH/CRITICAL-Aktionen benötigen die CAPITAL-AI Capability-/Approval-/Step-up-Kette;
5. Agenten dürfen ihre eigenen HIGH/CRITICAL-Änderungen nicht autorisieren;
6. Break-glass muss attributierbar, zeitlich begrenzt und auditierbar sein.

Sobald eine unabhängige Reviewer-Identität verfügbar ist, werden mindestens eine Approval sowie Code-Owner-Review und Last-Push-Approval aktiviert.

Gewünschte weitere GitHub-Controls

Ohne Single-Owner-Deadlock sollen serverseitig gelten:

- `required_review_thread_resolution = true`
- `require_code_owner_review = true`
- `require_last_push_approval = true`
- stale review dismissal bei neuen relevanten Pushes
- dediziertes geschütztes GitHub Environment `production`
- Deployment nur von `main`

Die aktuell angebundene GitHub-App bietet keine Ruleset-/Environment-Schreiboperation. Diese serverseitigen Felder sind daher als expliziter Admin-Handoff zu behandeln und dürfen nicht fälschlich als automatisiert umgesetzt dokumentiert werden.

Agent Branch Contract

Zugelassene Präfixe:

- `agent/`
- `claude/`
- `gemini/`
- `copilot/`
- `ai/`

Der Providername ist nur Herkunftsmetadatum. Autorisierung erfolgt ausschließlich über Capability, Risk Class, Policy und Approval.

Geschützte Artefaktklassen

CODEOWNERS soll mindestens folgende sicherheitskritische Bereiche abdecken:

- GitHub Workflows und Policies
- Agent Tooling und Control Plane
- IAM / Step-up / Capability Controls
- Telemetry / Audit / Traceability
- Deployment Identity / Render-Konfiguration
- Supabase Migrationen
- Release-/Supply-Chain-Evidence
- ADR-/ESS-Governance für diese Controls

Drift Contract

Der maschinenlesbare Sollvertrag liegt in:

`.github/policies/main-production-protection.expected.json`

Serverseitige Evidence soll in späteren Assurance-Läufen gegen diesen Sollvertrag verglichen werden.

Definition of Done M1

M1 ist abgeschlossen, wenn:

- der Sollvertrag versioniert ist;
- CODEOWNERS die Agent-/IAM-/Telemetry-/Supply-Chain-Flächen abdeckt;
- Single-Owner-Risiko und Promotion-Bedingung dokumentiert sind;
- kein bestehender Ruleset-Schutz abgeschwächt wird;
- serverseitig nicht automatisierbare Einstellungen als Handoff ausgewiesen sind;
- PR-CI erfolgreich ist.

GitHub Pro Enablement Decision — Documentation Refresh

Quelle: docs/governance/GITHUB_PRO_ENABLEMENT_DECISION.md

GitHub Pro Enablement Decision — Documentation Refresh

Status: DEFERRED / NOT A DEVELOPMENTCHAIN BLOCKER Date: 2026-08-11

PR #190 mixed GitHub plan/cost decision material with production deployment implementation. PR #192 supersedes that approach and retains only architecture-relevant documentation.

Current invariant: the existing repository ruleset and required check `build-and-test` remain authoritative. No paid-plan feature may become an undocumented prerequisite. Any future GitHub plan upgrade requires a separate cost/benefit decision and may not alter the M2 documentation-first gate.

GROK Post-PR-Merge Continuation Policy

Quelle: docs/governance/GROK_POST_PR_MERGE_CONTINUATION_POLICY.md

GROK Post-PR-Merge Continuation Policy

Document ID: GOV-GROK-POST-MERGE-001

Status: ACTIVE

Date: 2026-08-16

Previous: PROPOSED 2026-08-15

Repository: SvenKulessa/Finance

Authority:

- docs/governance/DEVELOPMENT_CHAIN_EXECUTION_POLICY.md
- docs/governance/DEVELOPMENT_CHAIN_BRANCH_LIFECYCLE_POLICY.md
- docs/governance/HUMAN_OWNER_PR_APPROVAL_POLICY.md
- ADR-0039, ADR-0069 (Nachtrag 2026-08-16), ADR-0073
- CAPITAL-AI DevelopmentChain (fail-closed)

Accountable Owner: SvenKulessa

Applies to: Grok (xAI) und vergleichbare Agenten mit Schreib-/PR-Capability im Finance-Repository

1. Zweck

Diese Richtlinie definiert das **verbindliche Verhalten von Grok nach Erstellung eines Pull Requests im Chat**.

Sie stellt sicher, dass:

1. Grok **niemals** selbst merged (MERGE bleibt Human/Owner-only).
2. Nach PR-Erstellung im Chat ein klarer **Wartezustand** eingenommen wird und auf den Merge **über Trigger / Automation / Session-Fortsetzung** gewartet wird.
3. Nach erfolgreichem Human-Merge die **codebasierte erfolgreiche Einsetzung** (CI, Deployment / Live-Status) geprüft wird.
4. Bei positivem Ergebnis **konkrete Vorschläge** gemacht werden, wie im Chat weitergearbeitet werden soll.
5. Bei negativem oder unklarem Ergebnis **Korrekturen** vorbereitet und als neues, scoped Workitem behandelt werden.
6. Evidence, Roadmap- und Traceability-Sync sowie Branch-Cleanup konsequent erfolgen.

Hinweis 2026-08-16: Owner-Checkboxen und Review ☐ / okay sind als CI-Gate **retired**. Ab M10 gilt Passkey-Autorisierung.

2. Geltungsbereich und Nicht-Ziele

Gilt für

- Jeden von Grok erstellten oder maßgeblich betriebenen PR gegen `main`.
- Klassen D/C/R und PRs, die Klasse-M-Mutationen vorbereiten.
- Session-übergreifende Fortsetzung und Merge-Watch-Automations.

Gilt nicht für

- Human-erstellte PRs (Beobachtung erlaubt, keine autorisierte Fortsetzungsinstanz).
- Direkte Produktionsmutationen ohne REM + Owner Mutation Approval.
- Self-Merge oder automatisierte Merge-Versuche.

3. Verbindliche Phasen-Sequenz

```
PR CREATED im Chat (Template-Contract ≥ 1.4.0 + Claim + Baseline)
↓
[PHASE A] WAIT FOR HUMAN MERGE (Chat-Wartezustand + Trigger/Automation)
↓
Human Merge auf main erfolgt
↓
[PHASE B] POST-MERGE TECHNICAL VERIFICATION
↓
[PHASE C] DEPLOYMENT / CODE-BASED SUCCESS CHECK
↓
[PHASE D] DECISION + CONTINUATION IM CHAT
```

4. Phase A — Wait for Human Merge (Chat-Trigger)

Regeln

1. Nach `create_pull_request` **stoppt** Grok mutierende Arbeit am selben Workitem.
2. Grok wartet auf die **explizite menschliche Merge-Entscheidung**.
3. Erlaubt während der Wartephase: - Read-only Status-Abfragen (PR-State, Checks, mergeable_state). - Berichte an den Owner. - Reparatur von CI-/Template-Contract-Fehlern **innerhalb des Work-Claim-Scopes** (kein Checkbox-/Emoji-Gate mehr).
4. **Verboten:** Merge ausführen; Force-Push auf geschützte Branches ohne Autorisierung; Scope-Erweiterung ohne neuen Claim; Druck-Aktionen („bitte mergen“).

Trigger

1. Session-Kontinuität (User: „PR-Status prüfen“, „fahre fort“, ...).
2. Optional: scheduled Grok Automation für Merge-Watch.
3. Bestehende stündliche Finance PR-Governance-Prüfung.

Sobald `merged == true` → Phase B.

5-7. Phase B/C/D

Unverändert in Substanz: Post-Merge-SHA/CI/Branch-Delete/Claim; Deploy/Health-Verifikation je Klasse; Chat-Fortsetzung oder Korrektur-Workitem. Korrektur-PRs durchlaufen den **vereinfachten** Zyklus (kein Owner-Checkbox-Gate).

8. Agenten-Grenzen

Darf	Darf nicht
PR erstellen	MERGE
CI/Deploy lesen	Self-Approve HIGH/CRITICAL
Evidence schreiben	Secrets in Evidence/PR-Body
Branch nach Merge löschen (Policy)	Alten Branch wiederverwenden
Next Steps im Chat vorschlagen	Ohne Claim Scope erweitern
Merge-Watch Automation	Automatisch mergen

9-12. Automations, Exit Criteria, Änderungsprozess, Kurzreferenz

Automations dürfen nur berichten und planen, niemals mergen. Exit Criteria: Human merge, merge SHA, Required Checks, Branch-Delete, Claim released, Deploy-Verifikation (oder D-only), Evidence, Roadmap-Sync, Chat-Next-Step.

```
PR erstellt → STOP mutierend → warte Human Merge
Merge erkannt → Phase B/C → VERIFIED PASS → Evidence + Next Steps im Chat
               → FAILED → neuer Fix-Branch/PR
```

Fail-closed. Unklare Deployment-Lage = kein Fortschritt.

Ende GOV-GROK-POST-MERGE-001 — ACTIVE 2026-08-16 (Owner-Gate ritual retired)

GROK PR Project-Context Tagging Policy

Quelle: docs/governance/GROK_PR_PROJECT_CONTEXT_TAGGING_POLICY.md

GROK PR Project-Context Tagging Policy

Document ID: GOV-GROK-PR-CONTEXT-001

Status: PROPOSED

Date: 2026-08-16

Repository: SvenKulessa/Finance

Authority:

- docs/governance/DEVELOPMENT_CHAIN_EXECUTION_POLICY.md
- docs/governance/HUMAN_OWNER_PR_APPROVAL_POLICY.md
- docs/governance/GROK_POST_PR_MERGE_CONTINUATION_POLICY.md (GOV-GROK-POST-MERGE-001)
- ADR-0039, ADR-0069
- CAPITAL-AI DevelopmentChain (fail-closed)

Accountable Owner: SvenKulessa

Applies to: Grok (xAI) und vergleichbare Agenten mit Schreib-/PR-Capability, die Pull Requests im Finance-Repository (oder parallel betriebenen Capital-AI-Repositories) erstellen

1. Zweck

Diese Richtlinie stellt sicher, dass **jeder von Grok erstellte Pull Request** den **Ursprungs-Kontext** (Grok-Projekt / Ordner / Chat-Session-Kontext) klar und maschinen- sowie menschenlesbar trägt.

Ziel:

1. Sofortige Identifizierbarkeit, aus welchem Grok-Projekt / welcher Chat-Session der PR stammt.
2. Vermeidung von Kontext-Vermischung bei parallelen Agent-Arbeiten (Grok, Claude, andere).
3. Bessere Traceability zwischen Chat-Auftrag, Work-Claim und PR.
4. Unterstützung der Multi-Projekt-/Multi-Folder-Arbeit (z. B. Finance-Core, SEO, Frontend, Governance).

Beispiel-Titel:

```
[Capital-AI SEO] docs(governance): add GROK PR Project-Context Tagging Policy
[CAPITAL-AI Governance] docs(governance): update document-registry
[Finance] feat(api): improve healthz response
```

2. Geltungsbereich

Gilt für

- Jeden Pull Request, den Grok (oder ein vergleichbarer Agent unter Grok-Steuerung) **erstellt**.
- Alle Repositories, in denen Grok im Rahmen von Capital-AI / SvenKulesa-Projekten PRs öffnet (primär `SvenKulesa/Finance`).
- Sowohl Klasse-D- als auch C/R/M-PRs.

Gilt nicht für

- Human-erstellte PRs (Empfehlung, aber keine Pflicht).
- PRs, die Grok nur kommentiert oder beobachtet, aber nicht selbst angelegt hat.
- Interne Draft-Branches ohne PR.

3. Verbindliche Regel (fail-closed)

Jeder von Grok erstellte Pull-Request-Titel MUSS mit einem Project-Context-Tag in eckigen Klammern beginnen.

Format

```
[<Project-Context>] <conventional-type>(<scope>): <kurze Beschreibung>
```

- Der Tag steht **ganz am Anfang** des Titels.
- Der Tag ist in eckigen Klammern `[...]`.
- Der Tag-Inhalt ist der **Name des Grok-Projekts / Ordners / Chat-Kontexts**, in dem der Auftrag zur PR-Erstellung erteilt bzw. die Session geführt wurde.
- Keine Leerzeichen unmittelbar nach der öffnenden Klammer; nach der schließenden Klammer genau ein Leerzeichen vor dem Rest des Titels.
- Der Tag darf nicht leer sein und sollte stabil und wiedererkennbar sein (keine flüchtigen Session-IDs).

Ableitung des Tags

Grok leitet den Tag aus dem **aktuellen Projektkontext** ab:

1. Primär: Name des Grok-Projekts, in dem der Chat geführt wird.
2. Alternativ / bei Mehrdeutigkeit: explizit vom Owner im Chat genannter Projekt- oder Ordnername.
3. Fallback (nur wenn kein klarer Projektname verfügbar): `[CAPITAL-AI]` oder `[Finance]`.

Beispiele gültiger Tags:

- `[Capital-AI SE0]`

- [CAPITAL-AI Governance]
- [Finance]
- [capital-ai-frontend]
- [Systemadmin]
- [Marketing]

Verbotene Formen

- Fehlen des Tags
- Tag am Ende oder in der Mitte des Titels
- Runde Klammern, geschweifte Klammern oder andere Formate statt [...]
- Mehrere konkurrierende Tags ohne klare Primär-Zuordnung
- Generische Tags wie [Grok] oder [PR] ohne Projektbezug

4. Zusätzliche Empfehlungen (nicht blockierend)

1. PR-Body - Abschnitt 1 (Arbeitsauftrag)

Optional, aber erwünscht: den Project-Context-Tag auch im Body unter Zweck oder als eigene Zeile `**Project-Context:** [Capital-AI SE0]` aufführen.

2. Work-Claim

Wenn ein Work-Claim erstellt wird, sollte der Claim-Dateiname oder ein Metadaten-Feld den gleichen Kontext-Tag spiegeln.

3. Commit-Messages

Der Tag ist nur für den **PR-Titel** verpflichtend. Commit-Messages folgen weiterhin Conventional Commits ohne erzwungenen Projekt-Prefix.

4. Automation / Governance-Checks

Zukünftige Template-Contract- oder advisory Checks dürfen den Tag validieren (z. B. Regex `^[[^\]]+\]\s`). Bis dahin gilt die Regel als Agenten-Verhaltenspflicht (fail-closed auf Agenten-Seite).

5. Integration in den PR-Erstellungsprozess

1. Owner-Auftrag / Chat-Kontext klären
2. Aktuellen main-SHA holen + Branch sync (PR-Sync-Pflicht)
3. Project-Context-Tag aus Chat-/Projekt-Kontext ableiten
4. Branch anlegen
5. Änderungen committen
6. PR erstellen mit Titel = "[<Tag>] <restlicher Conventional-Titel>"
7. Vollständige Template 1.3.5 + Work-Claim + Baseline
8. Owner-Gate abwarten (GOV-GROK-POST-MERGE-001)

Grok darf **keinen** PR ohne den verpflichtenden Tag öffnen. Bei Unklarheit über den korrekten Tag muss Grok vor `create_pull_request` nachfragen oder den Fallback [CAPITAL-AI] verwenden und dies im Body dokumentieren.

6. Agenten-Grenzen

Darf	Darf nicht
Tag aus aktuellem Grok-Projekt-/Chat-Kontext ableiten	Tag weglassen oder „vergessen“
Tag im Titel erzwingen	Human-PRs nachträglich umbenennen ohne Auftrag
Bei Unklarheit nachfragen oder dokumentierten Fallback nutzen	Beliebige / irreführende Tags erfinden
Policy in zukünftigen D-Klasse-PRs verfeinern	Self-Merge oder CI-Bypass

7. Änderungsprozess

Diese Policy darf nur durch Human/Owner-autorisierten PR geändert werden.
Grok darf Verbesserungsvorschläge als Klasse-D-PR einbringen, muss dabei selbst den Project-Context-Tag gemäß dieser Richtlinie verwenden.

8. Kurzreferenz für Grok

```
Vor create_pull_request:  
→ Project-Context-Tag bestimmen (Grok-Projekt / Ordner / Chat-Name)  
→ Titel MUSS beginnen mit: [Tag]  
→ Beispiel: [Capital-AI SEO] docs(governance)[] ...  
  
Kein Tag [] kein PR.  
Fail-closed.
```

Ende GOV-GROK-PR-CONTEXT-001 — 2026-08-16

HUMAN / OWNER Pull Request Approval Policy

Quelle: docs/governance/HUMAN_OWNER_PR_APPROVAL_POLICY.md

HUMAN / OWNER Pull Request Approval Policy

Status: REQUIRED

Effective from: 2026-08-11

Updated: 2026-08-16

Repository Owner: SvenKulesa

Purpose

Pull requests targeting `main` remain human-visible. AI agents may prepare branches, commits, PRs, evidence and fixes. They **MUST NOT** self-approve or autonomously merge.

Stand 2026-08-16 (Owner-Entscheidung): Die frühere Pre-CI-Zeremonie mit PR-Body-Checkboxen und Current-Head-Review `/ okay` ist **aufgehoben**. Der Merge-Prozess ist vereinfacht. Ab Development-Chain-Punkt **M10** wird die starke CI-/Autorisierungsbindung über **Passkey/WebAuthn** hergestellt (siehe `docs/runbooks/M10_PASS-KEY_OWNER_PR_AUTHORIZATION.md`).

Was entfällt (retired)

Die folgenden Mechanismen sind **nicht mehr** Voraussetzung für technische CI oder Merge-Bereitschaft:

1. PR-Body-Checkboxen
 - Human/Owner: vollständigen PR-Diff geprüft.
 - Human/Owner: alle geänderten Dateien im Tab Files changed als Viewed markiert.
2. Current-Head-Owner-Review mit exakt `/` oder `okay`
3. Event-Snapshot-Regel, dass der letzte Checkbox-Edit teure CI auslöst
4. Maschinenlesbare IDs `CAPITAL_AI_OWNER_DIFF_ATTESTATION` / `CAPITAL_AI_OWNER_FILES_ATTESTATION` als CI-Gate

Diese Rituale dürfen in historischen PRs und Evidence referenziert bleiben, sind aber **nicht mehr normativ**.

Vereinfachter Ablauf (aktuell, vor M10-Cutover)

```
PR OPEN / UPDATE
→ Governance-Checks (Template-Contract, Workflow-Security, ...)
→ technische CI (build-and-test gemäß Checkklasse D/C/R/M)
→ Human/Owner entscheidet über Merge
→ Human Merge (explizite Anweisung; Agenten mergen nicht)
```

Empfohlen (nicht CI-blockierend): Owner liest den Diff unter Files changed vor dem Merge. Das ist gute Praxis, kein fail-closed Gate mehr.

Was bleibt verbindlich

1. **Human-/CODEOWNER-Merge-Freigabe:** Merge bleibt Human/Owner-only. Agenten und Connector-Clients dürfen CI-Erfolg **nicht** als Merge-Autorisierung interpretieren.
2. **Kanonischer PR-Template-Contract** (Version $\geq 1.4.0$): Abschnitte und Baseline-Marker bleiben; verkürzte Free-Form-Bodies sind unzulässig.
3. **Required Checks** gemäß Ruleset (u. a. `build-and-test`) bleiben fail-closed.
4. **Externe Produktionsmutationen** brauchen weiterhin separate Owner Mutation Approval (Klasse M / REM).
5. **Ab M10:** Passkey/WebAuthn-Transaction für `AUTHORIZE_PR_CI` (und ggf. weitere privilegierte Aktionen) laut M10-Runbook; bis dahin gilt der vereinfachte Ablauf oben.

Verbindlicher PR-Template-Contract

Jeder PR gegen `main` MUSS `.github/pull_request_template.md` verwenden.

1. Kein verkürzter oder frei formulierter Body anstelle der Vorlage.
2. Nummerierte Abschnitte bleiben; nicht Zutreffendes mit `N/A`.
3. Produktions-Baseline-Marker (`CAPITAL_AI_PRODUCTION_BASELINE_*`) bleiben maschinenlesbar.
4. Owner-Attestation-IDs und Checkbox-Gate-Texte sind **entfernt** und dürfen nicht wieder als CI-Pflicht eingeführt werden, außer über einen eigenen ADR + Shadow → Cutover (M10-Pfad).
5. `Human-/CODEOWNER-Freigabe für Merge erforderlich:** Ja` bleibt im Contract.

CI-Sequenz (vereinfacht)

- Teure Validierung startet bei normalen PR-Events (`opened` / `synchronize` / `reopened` / `ready_for_review`) gemäß Workflow.
- Kein `pull_request: edited`-only-Gate mehr für Owner-Checkboxes.
- Ein neuer Commit löst CI erneut aus; es gibt keine Head-gebundene Emoji-Review-Evidence mehr zu invalidieren.
- Erfolgreiche technische CI ist **keine** Merge-Autorisierung.

Owner authentication assurance

GitHub-Account-Reviews beweisen keine Passkey-/Gerätebindung. Für privilegierte Übergänge (Produktion, Break-Glass, Capability-Elevation) und ab **M10** für PR-CI-Autorisierung gilt die CAPITAL-AI WebAuthn/Passkey-Architektur. Bis M10-Cutover darf kein Workflow behaupten, GitHub-Review-Text beweise Passkey-Authentisierung.

AI-agent capability restriction

- AI agents MAY: READ, ANALYZE, PLAN, BRANCH, COMMIT, open/update PRs, inspect CI, propose fixes (gemäß aktivem Roadmap-Profil).
- AI agents MUST STOP before MERGE.
- Merge nur nach ausdrücklicher menschlicher Anweisung für den konkreten PR.
- Read-only Daily-Task-Agents: `AUTONOMOUS_AGENT_CONCEPT_GATE.md` (keine BRANCH/COMMIT/PR/MERGE-Capabilities).

Evidence (vereinfacht)

1. kanonische Template-Struktur im PR-Body;
2. sichtbarer Diff unter Files changed;
3. aktueller PR-Head-SHA;
4. grüne Required Checks / `build-and-test` gemäß Klasse;
5. menschliche Merge-Entscheidung + Merge-Commit;
6. ab M10 zusätzlich: Passkey-Approval-/Consumption-Evidence laut M10-Runbook.

Bezug

- `docs/runbooks/M10_PASSKEY_OWNER_PR_AUTHORIZATION.md`
- `docs/governance/DEVELOPMENT_CHAIN_EXECUTION_POLICY.md`
- `docs/adr/ADR-0069-human-owner-comment-gate-and-dispatched-pr-ci.md` (Übergang / Recovery; Checkbox-Gate retired)
- DevelopmentChain, Agent IAM, Merge-Governance

CAPITAL-AI Marketing Agent Roadmap Execution Policy

Quelle: docs/governance/MARKETING_AGENT_ROADMAP_EXECUTION_POLICY.md

CAPITAL-AI Marketing Agent Roadmap Execution Policy

Status: DRAFT / NOT ACTIVE Date: 2026-08-12 Owner: SvenKulesa Logical agent: `capital-ai-marketing-roadmap-executor` Authority: proposed ESS-0022 / ADR-0068

1. Purpose

This policy defines how the future Marketing Roadmap Executor may execute Owner-approved work without inheriting the broader authority of the Systemadmin Roadmap Executor.

Until this policy is Human-approved and technically enforced, it is documentation only and grants no mutation authority.

2. Trust model

The Marketing Agent is not a trust root.

Trust remains in:

- Human/Owner authorization;
- CAPITAL-AI Agent IAM / Policy Gate;
- exact mandate validation;
- execution-host identity;
- durable audit evidence;
- repository/platform target binding;
- content validation/provenance/approval;
- Human-only merge.

Provider identity (ChatGPT, Claude, Gemini or another model) is metadata only.

3. Marketing Roadmap Execution Mandate

The future Marketing subject SHALL use a dedicated mandate profile or a safely generalized REM contract that can distinguish agent subjects without wildcard inheritance.

A Marketing mandate must bind at minimum:

- mandate id;
- Human Owner actor;
- `subjectAgentId = capital-ai-marketing-roadmap-executor`;

- repository and base branch;
- Marketing Roadmap items;
- authority references;
- exact allowed capabilities;
- exact allowed paths;
- exact targets;
- maximum risk class;
- allowed/prohibited mutation classes;
- validity window;
- max open PRs;
- CI budget policy;
- kill switch;
- preflight requirements;
- evidence requirements;
- approval evidence reference when active.

A Systemadmin REM is never automatically valid for the Marketing Agent and a Marketing mandate is never automatically valid for the Systemadmin Agent.

4. Capability states

State MKT-0 — specification/read-only

Allowed:

READ, ANALYZE, PLAN

Denied:

BRANCH, COMMIT, PR, CI_REQUEST, DEPLOY_REQUEST, PRODUCTION_MUTATION, MERGE, SOCIAL_PUBLISH

State MKT-1 — bounded repository execution

May later allow, only under a valid Marketing mandate and verified execution host:

READ, ANALYZE, PLAN, BRANCH, COMMIT, PR, CI_REQUEST

Still denied by default:

DEPLOY_REQUEST, PRODUCTION_MUTATION, SOCIAL_PUBLISH, MERGE .

State MKT-2 — controlled external marketing mutation

Not authorized by this policy draft.

Requires a later ADR, exact capability/target enforcement, strong approval binding, idempotency, rollback/postcondition verification and production evidence.

5. Domain path policy

Candidate Marketing-owned paths:

- `src/platform/SocialMediaEngine/**`;
- `server/socialMedia/**`;
- `docs/seo/**`;
- `src/platform/SeoEngine/**` after creation;
- Marketing-specific tests/evidence;
- Marketing-specific architecture/governance artifacts explicitly named in the mandate.

Shared Integration Zone:

- `AGENTS.md`;
- root package/lock files;
- `.github/workflows/**`;
- global IAM/Security/Compliance;
- global Event Mesh/Traceability registries;
- app/server bootstrap;
- deployment descriptors;
- shared database migrations;
- version/release sources of truth.

A shared-zone change is not implied by a Marketing path prefix. It requires explicit mandate scope and conflict coordination.

6. Mandatory preflight

Before each future mutating work package:

1. resolve current `main` SHA;
2. resolve current Marketing Roadmap phase and dependency gates;
3. validate the Marketing mandate and subject id;
4. inspect open PRs and changed-file overlap;
5. inspect relevant ESS/ADR/code/evidence;
6. classify risk and mutation class;
7. identify shared integration resources;
8. define required tests and negative tests;
9. define rollback/abort path;
10. define CI cost class;
11. verify no Owner-only action is required;
12. verify audit sink and execution-host prerequisites.

Security-critical ambiguity fails closed.

7. Repository mutation flow

Future MKT-1 flow:

```
validated Marketing mandate -> durable authorization audit -> audit-bound permit -> fresh  
branch -> exact file action -> outcome audit -> commit -> PR -> Human review/CI -> Human  
merge -> branch deletion .
```

No repository side effect may occur when the permit/audit step fails.

8. Pull Request governance

Before verified Marketing standing-authorization enablement, ADR-0039 remains fully applicable: every new Marketing PR requires explicit Human authorization after scope/risk/drift/overlap summary.

After a future Human-approved Marketing mandate and verified technical enforcement, a mandate MAY count as standing PR-creation authority only for PRs fully inside its exact boundaries.

In all states:

- merge is Human-only;
- current-head Human review is required;
- required repository attestations/checks remain authoritative;
- a new commit invalidates stale review evidence where repository policy says so.

9. Branch lifecycle

Each work package uses a fresh branch from current `main`.

Recommended prefix:

```
agent/marketing/<roadmap-item>-<short-name>
```

or where GitHub branch naming constraints/workflow conventions require:

```
agent/marketing-<roadmap-item>-<short-name> .
```

After successful Human merge, the remote branch MUST be deleted. Closed/superseded branches are deleted after necessary evidence retention. Merged branches are never reused.

10. Content-generation boundary

Generation and publication are distinct capabilities.

The agent may plan/generate content only through approved content contracts. Retrieved web/news/SEO content is untrusted data and cannot issue tool commands or alter policy.

Quantitative financial statements require attributable source evidence. Unsupported or synthetic claims fail closed for public marketing use.

11. Human content approval

Generated financial marketing content requires Human approval until a later approved ADR explicitly defines a safe autonomous publication class.

Approval should bind:

- content package id;
- content hash;
- final asset hashes;
- target platforms;
- approver;
- issuance/expiry;
- single-use/replay semantics where appropriate.

Any material content, asset or target-platform mutation invalidates approval.

12. External providers and credentials

LLMs, image models, TTS providers and video renderers receive least-privilege task data only.

They MUST NOT receive:

- Social OAuth refresh/access tokens unless an explicit server-side provider integration requires a scoped token and the model cannot observe it;
- GitHub credentials;
- Supabase privileged keys;
- Stripe secrets;
- Owner sessions;
- TOTP/passkey material;
- unrestricted Render/IONOS credentials.

Media workers are content processors, not publishers or infrastructure principals.

13. Social publishing boundary

The existing CAPITAL-AI SocialMediaEngine remains the distribution authority.

Third-party media-generator publishing features MUST be disabled/not used by default.

Future external publishing requires:

- approved content package;
- approved final asset;

- connected user-scoped account;
- target-platform authorization;
- idempotency/replay protection;
- publish audit/log;
- post-publish result verification;
- explicit handling of provider `pending/review` states.

14. Kill switch and STOP conditions

Owner can revoke a Marketing mandate at any time.

Automatic STOP:

- mandate expired/revoked;
- wrong agent/repository/path/target;
- scope drift;
- self-elevation/self-approval;
- shared-resource collision without approved resolution;
- audit persistence unavailable;
- security/consent control regression;
- unverified financial source claim;
- approval mismatch;
- secret exposure;
- unsupported external mutation;
- repeated verification failure;
- rollback/postcondition cannot be proven.

15. Audit contract

Every future mutating action must correlate at least:

`mandateId + roadmapItem + humanActor + agentId + client/app + session/request + capability + risk + target + branch/commit/PR` where applicable + `policy decision + auditReference + result`.

Content operations additionally correlate:

`contentPackageId + sourceDigest + contentHash + assetHashes + approvalId + targetPlatforms + publishLogIds` where applicable.

Security audit evidence and business content state remain separate persistence domains.

16. CI policy

Use the repository CI budget model:

- docs-only fast path;
- targeted unit/contract tests before expensive CI;
- no real GPU/video render for every PR;
- mock/fixture provider tests for adapter code;
- one final expensive CI for the reviewed PR head;
- release/preproduction evidence for real external rendering/publishing where required.

17. Relation to Systemadmin Agent

The Systemadmin Agent owns cross-cutting execution infrastructure; the Marketing Agent owns Marketing domain work.

If a Marketing task requires shared/global control-plane or CI changes, the preferred model is:

Marketing Agent proposes exact integration requirement -> Human/coordination gate -> Systemadmin or explicitly authorized integration writer changes shared control -> Marketing Agent continues domain work.

This avoids cross-domain privilege creep.

18. Activation rule

This policy becomes executable authority only after:

- ESS-0022 and ADR-0068 are accepted;
- the canonical registry/traceability entries are updated;
- required SA host foundations are VERIFIED PASS;
- Marketing subject/mandate enforcement and negative tests are VERIFIED PASS;
- Human/Owner approves the first bounded Marketing mandate.

Pull Request Check Classification

Quelle: docs/governance/PR_CHECK_CLASSIFICATION.md

Pull Request Check Classification

Status: ACTIVE

Updated: 2026-08-16

Authority: CAPITAL-AI DevelopmentChain

Zweck

Dieses Dokument legt fest, welche Checks ein Pull Request abhängig vom tatsächlichen Änderungsumfang durchlaufen muss. Ziel ist, unnötige CI-Läufe zu vermeiden, ohne Sicherheits- oder Qualitätsgates zu schwächen.

Stand 2026-08-16: Das frühere Pre-CI-Owner-Gate (Checkboxen + Review ☐ / okay) ist **retired**. Technische CI startet ohne diese Zeremonie. Merge bleibt Human/Owner-only. Ab M10 gilt Passkey-Autorisierung laut M10-Runbook.

Begriffe

Threat Model

Ein Threat Model beschreibt Assets, Trust Boundaries, mögliche Angreifer/Fehlbedienungen, Angriffswege, Auswirkungen und Gegenmaßnahmen. Es ist vor allem erforderlich, wenn ein PR Authentisierung, Autorisierung, Agent-Capabilities, Secrets, externe Schreibzugriffe, neue Trust Boundaries oder Produktionsmutationen einführt.

Negative Tests

Negative Tests prüfen nicht den gewünschten Happy Path, sondern beweisen, dass verbotene Zustände fail-closed abgelehnt werden. Beispiele: falscher Actor, fehlende Capability, falscher PR-Head-SHA, Replay, abgelaufene Approval-Evidence, falsche WebAuthn Origin/RP-ID, unerlaubter Produktionszugriff oder ungültige Konfiguration.

Rollback-Runbook

Ein Rollback-Runbook definiert, wie nach einer fehlerhaften Mutation der letzte verifizierte Zustand wiederhergestellt wird. Es enthält Trigger, Verantwortlichkeit, Rücksetzschrte, Daten-/Konfigurationsfolgen und Verifikation. Bei reinem Repository-Code kann `git revert` genügen. Externe Mutationen an Supabase, Stripe, Render, Credentials, Deployments oder Schemas benötigen ein explizites Runbook.

Checkklassen

D — Documentation-only

Nur `docs/**`, `.ai/**` oder Markdown.

Pflicht: Governance/Security, Docs-Fast-Path, `build-and-test`.

Nicht erforderlich: npm, TypeScript, Unit Tests, Production Build, Docker. Kein Owner-Checkbox-/Emoji-Gate.

C — Application / Test / Configuration

Anwendungs-/Servicecode, Tests oder nicht-dokumentarische Konfiguration ohne Runtime-/Deployment-Relevanz.

Pflicht zusätzlich zu Governance: Git-/Toolchain-Integrität, `npm ci`, Production Dependency Audit, Production Config Invariants, Docker-Hardening-Policy-Check, TypeScript/Lint, Unit Tests, Production Build, CSP-Test, Predeploy-Check, `build-and-test`.

R — Runtime / Dependency / Docker / Deployment

Dockerfile, Dependency-Manifeste, Server/Runtime, Render-Konfiguration, Runtime-/Docker-Security oder relevante Workflows.

Pflicht: vollständige Klasse C plus Docker Image Build, Image User/CMD/Healthcheck, Workflow Security bei Workflow-Änderungen sowie Deployment-/Rollback-Nachweis wenn Produktionsverhalten betroffen ist.

M — External Platform Mutation

Geplante Mutation an Supabase, Stripe, Render oder anderer produktionsverbundener Plattform.

Pflicht zusätzlich: autorisierende Roadmap/ADR/ESS, Owner Mutation Approval, Pre-Mutation Baseline/Test, ausführbares Rollback-Runbook, protokollierte Mutation, Post-Mutation Verification und Evidence. Der nächste Roadmap-Schritt bleibt bis `VERIFIED PASS` blockiert.

Auswahlregel

Die strengste zutreffende Klasse gilt für den gesamten PR. Ein neuer Commit, der den Scope erweitert, kann die Checkklasse erhöhen.

Merge-Regel

Ein PR ist merge-fähig, wenn:

1. alle Checks der gewählten Klasse PASS sind;
2. `build-and-test` PASS ist;
3. Governance PASS ist;
4. keine offenen merge-blockierenden Funde bestehen;
5. eine separate ausdrückliche menschliche Merge-Anweisung vorliegt (Agenten mergen nicht).

Nicht erforderlich: Owner-Body-Checkboxes, Review-Text `/ okay`, Viewed-Attestation als CI-Gate.

Ab M10: zusätzliche Passkey-/WebAuthn-Autorisierung gemäß `docs/runbooks/M10_PASS-KEY_OWNER_PR_AUTHORIZATION.md` .

Systemadmin Agent Roadmap Execution Policy

Quelle: docs/governance/SYSTEMADMIN_AGENT_ROADMAP_EXECUTION_POLICY.md

Systemadmin Agent Roadmap Execution Policy

Status: PROPOSED Date: 2026-08-12 Authority: ESS-0021, ADR-0065, ESS-0019, ADR-0058, HUMAN_OWNER_PR_APPROVAL_POLICY.md Accountable Owner: SvenKullessa

1. Purpose

This policy introduces a privileged **Systemadmin Roadmap Executor** profile for CAPITAL-AI. The purpose is to let an approved agent execute larger, coherent Roadmap work packages autonomously inside ChatGPT/connected execution clients without requiring a new human authorization for every branch, commit or Pull Request.

The profile is not an unrestricted administrator. Authority is delegated through a bounded, expiring, auditable **Roadmap Execution Mandate (REM)**.

The target operating model is:

OWNER-APPROVED ROADMAP MANDATE → READ/ANALYZE → SECURITY PREFLIGHT → BRANCH → IMPLEMENT → TEST → COMMIT → PR → OWNER REVIEW → CI → HUMAN MERGE

2. Systemadmin principal

Canonical logical agent id:

capital-ai-systemadmin-roadmap-executor

Provider/model identity is metadata only. ChatGPT, Claude or another client may host the profile, but the authority derives only from a valid REM plus the provider-neutral Agent IAM/Control Plane.

The execution principal MUST remain attributable through:

human_actor → client/app → agent → session → request → mandate → capability → target → decision → result → evidence

Credentials remain in the connector/tool host. Raw reusable credentials MUST NOT be exposed to the model.

3. Roadmap Execution Mandate (REM)

A REM is a Human/Owner-approved standing authorization for one bounded Roadmap segment. It replaces repeated per-PR creation prompts only within its exact scope.

The following single-use section rule applies to every human or agentic administrator principal. A mutation grant is valid for exactly one named Roadmap section/work package and

exactly one bound administrator principal. It MUST bind approval id, current `main` SHA, branch, targets, enumerated operations, risk ceiling, expiry, pre/post checks, rollback and audit path. The Execution Host MUST atomically consume the grant before the first side effect. Success, failure, cancellation or rollback permanently consumes it; retry requires a new Owner approval. Replay, concurrent consumption or any principal/scope/SHA/target/operation/risk/time drift is `DENY`.

A valid REM MUST define at minimum:

- immutable `mandateId` ;
- Owner actor id;
- subject agent id;
- repository and base branch;
- exact Roadmap/ESS/ADR references;
- allowed paths and target resources;
- allowed capabilities;
- maximum risk class;
- explicitly permitted mutation classes;
- prohibited/reserved mutation classes;
- validity window / expiry;
- maximum simultaneous branches/PRs;
- CI-budget policy reference;
- precondition/security checks;
- required tests;
- rollback strategy;
- kill-switch / revocation procedure;
- audit/evidence requirements.

A mandate is invalid when expired, revoked, its Roadmap state changed materially, the branch base is stale in a security-relevant way, or the requested action exceeds any declared scope.

4. Capabilities under an active REM

The Systemadmin Agent MAY receive the following capabilities without a new per-action Human prompt when they are explicitly listed in the active REM:

- `READ`
- `ANALYZE`
- `PLAN`
- `BRANCH`
- `COMMIT`

- `PR`
- `CI_REQUEST`

This permits the agent to take an approved Roadmap work package from current-state analysis through a review-ready Pull Request, including multiple scoped commits and automated remediation of technical failures before final Owner review.

`MERGE` is not and **MUST NOT** become an Agent IAM capability.

A section grant may contain multiple operations only when every operation is fully enumerated before approval. It is not a reusable standing administrator session and cannot authorize newly discovered follow-up work.

5. Repository mutation authority

Repository mutations are the default autonomous mutation class for this profile.

The agent **MAY** autonomously:

- create a fresh work branch from current `main`;
- change files covered by the REM;
- create/update tests and Evidence;
- commit scoped changes;
- update its own PR while the final Human/Owner review has not yet been issued;
- create a Draft or review-ready PR when the work package exit criteria are met;
- inspect CI and repair failures that remain inside the approved scope;
- synchronize Roadmap/ADR/ESS/Evidence inside the same approved work package.

A scope expansion beyond the REM is a **STOP** condition and requires a new or amended Owner-approved mandate.

6. External platform mutation authority

External production mutation is **not implied** by repository-write authority.

A REM **MAY** explicitly grant `PRODUCTION_MUTATION` only for a named, bounded and reversible mutation whose exact target and mutation class are declared before execution and whose Control-Plane implementation can technically enforce the mandate.

Before any delegated production mutation, all of the following **MUST** be true:

1. the Roadmap/ADR/ESS explicitly requires the exact mutation;
2. the REM explicitly includes `PRODUCTION_MUTATION`, target and mutation class;
3. the mandate approval used strong Owner step-up once the M10/WebAuthn approval service exists; until then such delegated production mutation remains disabled unless a separately accepted interim ADR says otherwise;
4. a read-only pre-mutation baseline passes;

5. the change is bounded, reversible and idempotent/replay-protected where applicable;
6. rollback is executable before mutation;
7. post-mutation verification is deterministic;
8. audit evidence records mandate, actor, agent, request, target, result and rollback state.

Until the technical Control Plane supports REM-bound production authorization, the Systemadmin Agent's autonomous mutation authority is limited to repository/branch/PR state and explicitly non-production environments.

7. Non-delegable Owner actions

The following remain Human/Owner-controlled even when a REM exists unless a future dedicated ADR explicitly changes one item with equivalent or stronger authentication:

- Pull Request merge;
- weakening branch protection, CODEOWNERS, required checks or Human/Owner gates;
- Owner/admin role elevation or IAM policy weakening;
- Owner MFA reset, recovery/break-glass activation or factor removal;
- secret/API-key/credential disclosure or unrestricted credential rotation;
- destructive production database operations or bulk user-data deletion;
- live billing/price/subscription/credit mutations that can directly change customer money or entitlement;
- production resource deletion;
- DNS/TLS/domain ownership changes;
- disabling security, audit, RLS, append-only or consent controls;
- expanding the agent's own mandate, capabilities, target set or expiry;
- reusing, transferring or replaying a consumed section approval.

The agent can prepare these changes and their runbooks, but it must stop before execution.

8. Mandatory security preflight

Before opening or mutating a work branch, the Systemadmin Agent **MUST** perform a read-only preflight:

1. resolve current `main` SHA and Roadmap state;
2. identify the next unblocked work package;
3. check open PR changed-file overlap;
4. inspect relevant code, ADR, ESS, Evidence and production state read-only where needed;
5. perform Deep Research/current best-practice review for security/architecture-sensitive work where relevant;
6. classify PR class D/C/R/M;

7. classify every requested capability/risk;
8. verify mandate validity and path/target scope;
9. verify no reserved Owner-only action is required;
10. define tests, negative tests, rollback and evidence;
11. estimate CI scope and avoid redundant full runs.

Any inconclusive security-critical precondition is DENY/STOP, not an invitation to guess.

9. Autonomous execution loop

Within a valid REM the agent MAY execute continuously:

```
ROADMAP READ → PREFLIGHT → BRANCH → IMPLEMENT → TARGETED TESTS → SECURITY SELF-CHECK →  
COMMIT → REPEAT UNTIL WORK PACKAGE COMPLETE → PR
```

After PR creation:

- the agent may repair CI/governance defects before final Human/Owner review;
- every new commit invalidates any earlier current-head Owner review;
- once the Owner has performed the final current-head review/Viewed attestations, the agent must not silently expand scope;
- merge remains a separate Human/Owner action.

10. CI and cost control

The Systemadmin Agent MUST respect the repository CI budget policy.

Default behavior:

- targeted local/read-only/static checks during implementation;
- no intentional repeated expensive full CI runs for the same unchanged head;
- one normal final `build-and-test` run after the Human/Owner gate unless a remediation commit makes revalidation necessary;
- do not rerun a known-invalid gate without changing the failing precondition.

11. Audit and evidence

Every autonomous mutating action MUST be correlated to the active mandate and, where the M5 audit path is available, record at least:

- `mandateId` ;
- Roadmap item/work-package id;
- human actor id;
- agent/client/session/request ids;
- capability;
- risk class;

- target resource;
- branch/commit/PR identifiers where applicable;
- authorization verdict;
- execution result;
- rollback status when applicable.

No secrets, raw tokens, full sensitive prompts or raw private request/response bodies belong in audit evidence.

12. Kill switch and revocation

Owner can revoke a REM at any time. Revocation immediately removes all mutating capabilities for the mandate.

Automatic STOP/revocation conditions include:

- mandate expiry;
- target/path scope mismatch;
- attempted reserved action;
- unexplained production drift affecting the work package;
- security-control regression;
- unresolvable concurrent-writer conflict;
- repeated verification failure;
- missing audit correlation;
- credential/identity ambiguity.

13. Branch lifecycle

Every Systemadmin work package uses a dedicated branch. A cloned repository/work branch is temporary execution state, not a persistent product artifact.

Mandatory lifecycle:

current main → new scoped branch → commits → PR → successful Human merge → branch deletion

After a successful PR merge into the Finance repository, the work branch MUST be deleted. A branch from a closed/superseded PR MUST also be removed after necessary Evidence has been retained. The agent MUST NOT reuse an old merged branch for a new Roadmap item.

14. Relationship to ADR-0039 and current PR governance

ADR-0039's per-PR creation authorization remains the default for normal interactive agents.

For the Systemadmin Agent only, an active Owner-approved REM is the PR-creation authorization for all PRs that remain inside the mandate. The agent does not ask again before each scoped PR.

This exception does **not** remove:

- Files-changed/Viewed Human review;
- current-head ☐ / okay Owner review;
- scope-appropriate CI;
- separate Human merge authority.

15. Enablement gate

This policy does not itself enable the agent.

Enablement requires:

1. ESS-0021 + ADR-0065 + Roadmap package merged;
2. Control-Plane support for REM validation where technical enforcement is required;
3. negative tests for scope, expiry, self-elevation, reserved actions and kill switch;
4. M5 audit correlation verified for Systemadmin actions;
5. Human/Owner approval of the first concrete REM.

Only then may the Systemadmin Agent operate without per-PR creation prompts inside that mandate.

Work Claim — ADR-0014 Phase 3.4.3

Quelle: docs/governance/work-claims/phase3-4-3-crypto-provider-chain.md

Work Claim — ADR-0014 Phase 3.4.3

Owner: server architecture decomposition workstream Status: active

Exclusive paths for this phase: - server/marketData/cryptoProviderChain.ts - tests/unit/cryptoProviderChain.test.ts - docs/architecture/phase3-4-3-crypto-provider-extraction.md

Protected shared paths not modified in this phase: - server.application.ts - server/marketData/marketDataCoordinator.ts - server/fmpIndices.ts - server/stockFundamentals.ts - src/lib/assetRegistry.ts - scoring services - Stripe/runtime/Render configuration

README.phase3 4 3

Quelle: docs/governance/work-claims/README.phase3-4-3.md

Phase 3.4.3 reserves only the crypto provider extraction files listed in `phase3-4-3-crypto-provider-chain.md`. Shared runtime and provider files remain protected from mutation until the separate cutover step.

Test Plan & Quantitative Finance Engine (QA Perspective)

Quelle: docs/qa/TEST_PLAN_AND_QA.md

Test Plan & Quantitative Finance Engine (QA Perspective)

Project: CAPITAL-AI
Auditor: QA & Quantitative Analysis Lead
Status: Certified & Production Ready

QA Methodology & Test Suite

Our Quality Assurance framework ensures that all algorithmic calculations, DCF models, and multi-factor scorings are mathematically robust, resilient to edge-case anomalies, and strictly conform to the **No-Demo-Data Policy**.

1. Quantitative Finance & Stock Scoring Engine (Warren Buffett Style)

We evaluate stocks as long-term business owners rather than short-term momentum traders, utilizing fundamental financial metrics:

Fundamental Scoring Framework:

- ROIC (Return on Invested Capital):** Stable and superior return rate over a 3-year trailing period (Weight: **20%**).
- ROE (Return on Equity):** Must exceed **15%** consistently (Weight: **15%**).
- Debt-to-Equity (D/E):** Must be below **0.5** to verify strict conservative capital structures (Weight: **15%**).
- FCF (Free Cash Flow):** Must represent consistent, non-volatile growth (Weight: **15%**).
- Operating Margin:** High and stable or improving margins indicating a strong brand (Weight: **10%**).
- Revenue Growth 3Y:** Positive and constant trailing growth (Weight: **10%**).
- P/E Ratio (Price-to-Earnings):** Fair valuation check; avoids over-hyped speculative sectors (Weight: **10%**).
- Moat-Qualität:** Qualitative assessment of pricing power, switching costs, and brand equity (Weight: **5%**).

Consolidated Buffett Score Classes:

- 90 - 100:** Very High Quality (Strong Buy Candidates)
- 75 - 89:** High Quality (Tradeable Watch)

- **60 - 74:** Good Quality (Observe / Hold)
 - **0 - 59:** Not Buffett-Typical (Strict Reject)
-

2. Trading Pattern Engine & Microstructure Analysis

For market entry and exit timing, our pattern recognition system checks technical and structural formations across 4 main classifications:

A. Bullish Reversal Patterns (Kurswirkung: Bullish)

- **Double Bottom** (Qualitätsklasse: 5)
- **Inverse Head and Shoulders** (Qualitätsklasse: 5)
- **Cup and Handle** (Qualitätsklasse: 5)
- **Falling Wedge** (Qualitätsklasse: 4)
- **Hammer / Bullish Engulfing** (Qualitätsklasse: 3)

B. Bearish Reversal Patterns (Kurswirkung: Bearish)

- **Double Top** (Qualitätsklasse: 5)
- **Head and Shoulders** (Qualitätsklasse: 5)
- **Rising Wedge** (Qualitätsklasse: 4)
- **Shooting Star / Bearish Engulfing** (Qualitätsklasse: 3)

C. Continuation Patterns (Kurswirkung: Continuation)

- **Bull Flag / Bear Flag** (Qualitätsklasse: 5)
- **Bull Pennant / Bear Pennant** (Qualitätsklasse: 5)
- **Ascending / Descending Triangle** (Qualitätsklasse: 4)

D. Neutral & Consolidation Patterns (Kurswirkung: Neutral)

- **Symmetrical Triangle** (Qualitätsklasse: 3)
 - **Volatility Squeeze / Coil Compression** (Qualitätsklasse: 4)
-

3. Mathematical Formula Test Scenarios

To avoid critical system failures in high-frequency computational environments, developers must run the following verification checks:

1. Graham Formula Safe Boundaries:

- **Test Case:** Input `EPS <= 0` or negative trailing values.
- **Expected Outcome:** The formula must catch the negative input gracefully, bypass square root calculations of negative values, and output a standardized safe floor value rather than returning `NaN` or a division by zero error.

2. Monte Carlo Iteration Limits:

- **Test Case:** Compute value-at-risk (VaR) with `Standard Deviation <= 0` or extremely high drift.
- **Expected Outcome:** The simulator must clamp standard deviation bounds, validate output array lengths, and verify that the geometric Brownian motion paths remain statistically plausible.

```
# TEST IMPLEMENTATION PATTERN
def test_graham_safe_boundaries():
    eps = -2.5
    growth_rate = 1.2
    # The calculation must clamp negative inputs and handle gracefully
    result = calculate_graham_value(eps, growth_rate)
    assert result >= 0.0
    assert not math.isnan(result)
```

Critical QA Deployment Checklist

- [] **React Hooks Isolation:** State updates are never called directly within the component rendering body to prevent infinite re-renders.
- [] **Stable Dependency Keys:** `useEffect` dependencies must only contain primitive keys (strings, numbers, booleans) or heavily memoized objects.
- [] **Touch Target Check:** Hit surfaces on all touch targets must be at least **44x44px** to align with BFGS accessibility guidelines.
- [] **Dynamic Array Guards:** Every client-side response mapping must be guarded with `Array.isArray(data)` to prevent client crashes.

CAPITAL-AI — Supabase Free-Tier Leaked Password Protection Exception

Quelle: docs/security/accepted-risk/SUPABA-SE_FREE_TIER_LEAKED_PASSWORD_PROTECTION_EXCEPTION_2026-08-02.md

CAPITAL-AI — Supabase Free-Tier Leaked Password Protection Exception

- **Record ID:** SEC-EXC-SUPABASE-AUTH-2026-08-02
- **Record Version:** 1.0.0
- **Date:** 2026-08-02
- **Environment:** Production
- **Platform Version:** 0.6.0
- **Status:** ACCEPTED — PLAN CONSTRAINT
- **Owner:** CAPITAL-AI Security / Platform Governance
- **Related ADR:** ADR-0031-supabase-free-tier-auth-security-compensating-controls.md
- **Advisor Finding:** auth_leaked_password_protection
- **Target State:** SUPABASE_SECURITY_ZERO_ACTIONABLE_WARNINGS

1. Finding

The production Supabase Security Advisor reports:

- **Name:** auth_leaked_password_protection
- **Level:** WARN
- **Description:** leaked-password protection is disabled.

Supabase documents the native leaked-password protection capability as available on **Pro Plan and above**. CAPITAL-AI currently operates the production Supabase project on the **Free Tier**.

Therefore this finding is not currently actionable within the active tariff.

2. Classification

Attribute	Decision
Security warning	Yes
Exploit automatically present	No
Native remediation available on Free Tier	No
Requires application-breaking workaround	No workaround authorized
Risk accepted permanently	No

Attribute	Decision
Risk accepted while Free Tier remains active	Yes
Mandatory remediation after Pro upgrade	Yes

Classification:

PLAN-CONSTRAINED / TEMPORARILY ACCEPTED

3. Current production advisor state

Following remediation of the Stripe-schema function search-path warnings, the current Security Advisor state contains:

- 1. `auth_leaked_password_protection` — `WARN` — plan-constrained.
- 2. `rls_enabled_no_policy` for `public.screening_slo_evidence` — `INFO` — intentional fail-closed/server-only architecture.

The following previously actionable findings were remediated:

- `stripe.set_updated_at` mutable search path — **FIXED**
- `stripe.set_updated_at_metadata` mutable search path — **FIXED**
- `stripe.check_rate_limit` mutable search path — **FIXED**

The three Stripe functions retain their existing behavior and now use a fixed empty function `search_path`.

4. Compensating controls

While the plan constraint remains active, CAPITAL-AI relies on layered controls including:

- verified Supabase identities;
- email confirmation;
- custom production SMTP;
- server-side IAM role verification;
- TOTP Step-Up for privileged owner operations;
- rate limiting;
- fail-closed RLS;
- server-only privileged keys;
- append-only security/audit evidence where defined;
- password policy hardening where configurable on Free Tier.

These controls reduce exposure but are **not represented as equivalent** to native leaked-password screening.

5. Prohibited responses to this finding

The following must not be done merely to silence the warning:

- weakening authentication requirements;
- disabling the Security Advisor;
- claiming a client-side password-strength meter performs leaked-password screening;
- storing a copied breach-password corpus in the production database without a separately reviewed architecture;
- routing plaintext passwords through custom application logging or observability;
- adding unrelated RLS policies to remove informational advisor findings.

6. Mandatory remediation trigger

This exception expires immediately when any of the following becomes true:

- the Supabase project is upgraded to Pro, Team or Enterprise;
- Supabase makes leaked-password protection available to Free Tier;
- a compliance requirement requires breached-password detection;
- an authentication/credential-stuffing incident requires immediate escalation.

On expiry, the next production release is blocked until:

1. Supabase native leaked-password protection is enabled;
2. Security Advisor is rerun;
3. `auth_leaked_password_protection` is absent;
4. signup and password-change/reset smoke tests pass;
5. this record is superseded or updated to `REMEDIATED` ;
6. the release acceptance record references the remediation evidence.

7. Acceptance statement

The current Free-Tier production state is accepted because all security findings that are actionable through the available database/security controls have been remediated, while the remaining WARN requires a higher Supabase plan according to Supabase documentation.

This acceptance does **not** downgrade the warning to PASS. It records why remediation is currently unavailable and defines the exact trigger that ends the exception.

Current security lifecycle status:

`SUPABASE_SECURITY_ZERO_ACTIONABLE_WARNINGS`

Developer Security Guidelines & Compliance Standards

Quelle: docs/security/SECURITY_GUIDELINES.md

Developer Security Guidelines & Compliance Standards

Project: CAPITAL-AI
Classification: Confidential / Internal Developer Scope

Security Pillars & OWASP Top 10 Governance

CAPITAL-AI is engineered using a "**Security-by-Design**" approach to mitigate risks outlined in the **OWASP Top 10** and enforce compliance with EU data protection regulations (DSGVO).

Vulnerability Category	Risk Level	Mitigation Architecture
A01:2021-Broken Access Control	High	Standardized JSON Web Tokens (JWT) mapped on PostgreSQL row-level security (RLS) policies.
A02:2021-Cryptographic Failures	Critical	Strict TLS 1.3 encryption on all channels. Absolute isolation of API keys server-side.
A03:2021-Injection	High	Parameterized queries using Express/Supabase. Input validation on all endpoints.
A05:2021-Security Misconfiguration	Medium	Whitelisted CORS configurations, secured cookies, disabled stack traces in client payloads.

Mandatory Security Rules for Developers

1. Absolute Secret Isolation (No Frontend Keys)

- Never embed API keys, secrets, or database credentials within the frontend source code. This applies directly to the `GEMINI_API_KEY`, `STRIPE_SECRET_KEY`, and `SUPABASE_SERVICE_ROLE_KEY`.
- Any environment variable used in client-side code must be prefixed with `VITE_` and be non-sensitive (e.g. `VITE_SUPABASE_URL`).
- Private credentials must remain purely on the backend, configured via server-side environment variables (`process.env.MY_SECRET`).

2. Defensive API Routing & CORS Whitelisting

- Disable permissive wildcard CORS bindings (`Access-Control-Allow-Origin: *`) in production environments. Explicitly authorize only verified domains.
- Rate limit all public endpoints to protect the system against Denial-of-Service (DoS) attacks and brute-force attempts.

3. Graceful Error Suppression & Quiet Payloads

Do not leak server architecture details, directory trees, library versions, or database errors in API response payloads. Log detailed stack traces on the server while returning clean, localized message models to the client.

```
// SECURE CODING PATTERN
try {
  const data = await executeSensitiveQuery();
  res.json({ success: true, payload: data });
} catch (error: any) {
  // Log the complete traceback with full debug info internally
  logger.error("[API Failure] Detailed audit trail:", error.stack);

  // Return an abstract, sanitised error string to the user
  res.status(500).json({ error: "Ein interner Verarbeitungsfehler ist aufgetreten." });
}
```

Security Proof-of-Concept (PoC) Verification

Every developer must verify their changes against our standard **Security Audit Suite**:

- **Authentication Check**: Sessions must time out after inactivity. Tokens must be stored in secure, `HttpOnly`, `SameSite` cookies or properly isolated headers.
- **Webhook Integrity**: Webhooks (such as `/billing/webhook` or `/api/stripe/webhook`) must verify signatures using the exact **raw body buffer**. Handlers must implement re-play-attack protection and be idempotent.
- **Input Sanitization**: Scrub all incoming parameters to prevent SQL injection, path traversal (`../` removal), and HTML tag injections (XSS).

CAPITAL-AI Security Remediation Baseline — 2026-08-12

Quelle: docs/security/SECURITY_REMEDIATION_BASELINE_2026-08-12.md

CAPITAL-AI Security Remediation Baseline — 2026-08-12

Status: DOCUMENTED / REMEDIATION REQUIRED Repository: `SvenKulessa/Finance` Verified repository baseline: `main@4767b86d0a183fef97ec812e77da3e5dcbd9d813` Audit basis: Security Audit 2026-08-12, findings F-01..F-18 Authority: `docs/roadmaps/DEVELOPMENT_CHAIN_ROADMAP.md`, existing ADR/ESS/governance controls and this baseline.

Purpose

This document reconciles the 2026-08-12 security findings with the current repository and the active DEVELOPMENT Chain. It prevents duplicate remediation work, distinguishes confirmed current defects from already-improved controls, and binds every finding to an implementation phase and verification gate.

No finding is considered closed merely because a roadmap phase mentions the topic. Closure requires current-code verification plus positive/negative test evidence on the exact remediation commit.

Repository observations verified during reconciliation

Confirmed open

- **F-01 — unauthenticated POST /api/docs-file :** `server/routes/documentationRoutes.ts` exposes the write handler without its own authentication/authorization gate. The runtime artifact guard is not an authorization boundary.
- **F-02 — production runtime artifact guard can fail open:** `server/runtime/runtimeArtifactGuard.mjs` accepts `CAPITAL_AI_RUNTIME_ARTIFACT_MODE=off`; the health logic can treat the disabled state as accepted. Production must instead fail closed.
- **F-11 — unauthenticated GET /api/docs-file :** the read handler is likewise not protected by an explicit identity/role decision.

Partially remediated / residual validation required

- **F-12 — fragile path sanitizer:** the current documentation route is stronger than the original audit description: it resolves the docs root with `path.resolve()`, rejects obvious traversal forms and checks the resolved target stays below the docs root. Therefore F-12 is not carried forward unchanged. S1 still replaces the remaining deny-list-oriented assumptions with a canonical root-containment helper based on `path.relative()` and adds symlink/encoded-path negative tests where applicable.
- **F-03 — CI self-modification risk:** M3 already introduced Owner/head-SHA gating, SHA-pinned/frozen controls and policy loading from trusted `main`. Residual risk remains

until workflow-control invariants are independently validated for `.github/workflows/**` changes.

- **F-14 — TOTP replay:** M5A is the authoritative remediation track. It may only become `VERIFIED PASS` after current-code and negative replay evidence; roadmap status alone does not close the audit finding.
- **F-17 — old Actions/Node runtime:** M3 already improved pinning. Runtime/action-version unification and supply-chain evidence remain M6 responsibilities.

Existing phases that already own remediation

- **F-10** unsigned/unverified supply-chain identity → M6 Supply Chain Provenance.
- **F-14** TOTP replay / privileged MFA semantics → M5A Native MFA/AAL2.
- **F-16** audit `token_id` correlation quality → M5 Audit/Telemetry.
- **F-17** action/runtime provenance and version consistency → M6.
- **F-18** deny-by-default RLS without an allow policy → retain as `ACCEPTED / INTENTIONAL` only where the table is intentionally unreachable; verify this invariant in database evidence rather than creating a permissive policy merely to silence the finding.

Finding reconciliation matrix

Finding	Severity	Current classification	Remediation owner	Required closure evidence
F-01	High	OPEN — confirmed	S1.1	anonymous write denied; authorized bounded write passes; audit evidence
F-02	High	OPEN — confirmed	S1.2	production boot/readiness fails if guard is disabled/misconfigured
F-03	High	PARTIAL / residual	S1.5	malicious workflow-change fixture cannot weaken required gate
F-04	High	VERIFY + REMEDIATE	S1.3	spoofed forwarding headers cannot alter canonical client identity/rate-limit key
F-05	High	VERIFY + REMEDIATE	S1.4	server secret resolver rejects client <code>VITE_*</code> aliases and build contains no server secrets
F-06	Medium	VERIFY + REMEDIATE	S1.6	enforced CSP passes application integration tests wi-

Finding	Severity	Current classification	Remediation owner	Required closure evidence
				thout unsafe-eval unless explicitly justified
F-07	Medium	VERIFY + REMEDIATE	S1.6	exactly one authoritative CSP path; legacy duplicate removed
F-08	Medium	VERIFY + REMEDIATE	S1.6	dynamic CORS emits correct Vary: Origin and rejects untrusted origins
F-09	Medium	VERIFY + REMEDIATE	S1.5	newline/output-injection fixture cannot alter GitHub outputs
F-10	Medium	PLANNED	M6	source/commit/artifact provenance is verifiable and bound to release
F-11	Medium	OPEN — confirmed	S1.1	anonymous read denied or explicit public allowlist proven
F-12	Low	PARTIAL — current code stronger than audit baseline	S1.1	canonical containment + traversal/encoding/symlink tests
F-13	Low	VERIFY + REMEDIATE	M9	token comparison uses constant-time semantics with negative tests
F-14	Low	M5A owns remediation	M5A	AAL2/TOTP replay negative test and current factor-state evidence
F-15	Low	VERIFY + REMEDIATE before horizontal scale	S1.3 / M7	shared or topology-safe rate limiter; multi-instance test where required
F-16	Low	M5 owns remediation	M5	non-reversible stable correlation identifier + redaction tests
F-17	Low	PARTIAL / M6	M6	

Finding	Severity	Current classification	Remediation owner	Required closure evidence
				runtime/action versions unified, pinned and evidenced
F-18	Info	ACCEPTED only if intentional deny-by-default is proven	M9 evidence	RLS evidence shows no unintended access path

VERIFY + REMEDIATE means the audit finding remains a blocking hypothesis until the current implementation is inspected in the implementation PR. It must not be silently marked closed from historical documentation.

Security invariants introduced by S1

1. Application authorization is enforced at the route/service boundary; runtime artifact controls are defense-in-depth only.
2. Production security controls fail closed. A missing or **off** security mode is a deployment/readiness failure, not a healthy state.
3. Proxy-derived metadata is evidence only after a configured trusted-proxy boundary; raw forwarded headers never establish identity.
4. Browser-visible **VITE_*** variables and server-secret namespaces are disjoint.
5. A pull request cannot weaken the mechanism that decides whether the same pull request may pass.
6. CORS/CSP policies have one authoritative implementation and explicit negative tests.
7. Findings close only with requirement → implementation → test → evidence traceability.

Required evidence structure

Each S1 implementation PR MUST record:

- audit finding IDs;
- exact base/head SHA;
- changed trust boundary;
- positive tests;
- negative/adversarial tests;
- CI result;
- deployment/runtime test when applicable;
- rollback condition;
- residual risk;
- final state: **OPEN** , **PARTIAL** , **VERIFIED PASS** , or **ACCEPTED** .

Evidence belongs under **docs/evidence/s1/** and must not contain live secrets, tokens, TOTP seeds, raw authorization headers or unredacted personal data.

Gate

M6 must not start while any High finding F-01..F-05 is still OPEN or PARTIAL .

Medium/Low findings may remain assigned to later canonical phases only when the owning phase, blocking dependency, exact verification requirement and residual risk are explicit. F-18 is never a reason to weaken deny-by-default RLS.